GTOS Detailed Volume - Volume 07 Fulfilment Supply Chain Domain Architecture

GTOS Enterprise Engineering Handbook - Final Edition 1.0

Source Chapters

- Supplier — Orientation and Evidence Baseline
- Supplier — Ubiquitous Language, Aggregates, and State
- Supplier — Commands, Events, APIs, Workflows, and Tools
- Supplier — Data, Tables, Migrations, Lineage, and Reconciliation
- Supplier — Portals, AI, Security, SLOs, Tests, and Operations
- Provider — Orientation and Evidence Baseline
- Provider — Ubiquitous Language, Aggregates, and State
- Provider — Commands, Events, APIs, Workflows, and Tools
- Provider — Data, Tables, Migrations, Lineage, and Reconciliation
- Provider — Portals, AI, Security, SLOs, Tests, and Operations
- Customer — Orientation and Evidence Baseline
- Customer — Ubiquitous Language, Aggregates, and State
- Customer — Commands, Events, APIs, Workflows, and Tools
- Customer — Data, Tables, Migrations, Lineage, and Reconciliation
- Customer — Portals, AI, Security, SLOs, Tests, and Operations
- CRM — Orientation and Evidence Baseline
- CRM — Ubiquitous Language, Aggregates, and State
- CRM — Commands, Events, APIs, Workflows, and Tools
- CRM — Data, Tables, Migrations, Lineage, and Reconciliation
- CRM — Portals, AI, Security, SLOs, Tests, and Operations
- Inventory — Orientation and Evidence Baseline
- Inventory — Ubiquitous Language, Aggregates, and State
- Inventory — Commands, Events, APIs, Workflows, and Tools
- Inventory — Data, Tables, Migrations, Lineage, and Reconciliation

- Inventory — Portals, AI, Security, SLOs, Tests, and Operations
- Warehouse — Orientation and Evidence Baseline
- Warehouse — Ubiquitous Language, Aggregates, and State
- Warehouse — Commands, Events, APIs, Workflows, and Tools
- Warehouse — Data, Tables, Migrations, Lineage, and Reconciliation
- Warehouse — Portals, AI, Security, SLOs, Tests, and Operations
- Shipping — Orientation and Evidence Baseline
- Shipping — Ubiquitous Language, Aggregates, and State
- Shipping — Commands, Events, APIs, Workflows, and Tools
- Shipping — Data, Tables, Migrations, Lineage, and Reconciliation
- Shipping — Portals, AI, Security, SLOs, Tests, and Operations
- Regulatory — Orientation and Evidence Baseline
- Regulatory — Ubiquitous Language, Aggregates, and State
- Regulatory — Commands, Events, APIs, Workflows, and Tools
- Regulatory — Data, Tables, Migrations, Lineage, and Reconciliation
- Regulatory — Portals, AI, Security, SLOs, Tests, and Operations
- Production Fulfilment — Orientation and Evidence Baseline
- Production Fulfilment — Ubiquitous Language, Aggregates, and State
- Production Fulfilment — Commands, Events, APIs, Workflows, and Tools
- Production Fulfilment — Data, Tables, Migrations, Lineage, and Reconciliation
- Production Fulfilment — Portals, AI, Security, SLOs, Tests, and Operations
- Allocation — Orientation and Evidence Baseline
- Allocation — Ubiquitous Language, Aggregates, and State
- Allocation — Commands, Events, APIs, Workflows, and Tools
- Allocation — Data, Tables, Migrations, Lineage, and Reconciliation
- Allocation — Portals, AI, Security, SLOs, Tests, and Operations
- Pick Pack Ship — Orientation and Evidence Baseline
- Pick Pack Ship — Ubiquitous Language, Aggregates, and State
- Pick Pack Ship — Commands, Events, APIs, Workflows, and Tools
- Pick Pack Ship — Data, Tables, Migrations, Lineage, and Reconciliation
- Pick Pack Ship — Portals, AI, Security, SLOs, Tests, and Operations
- Customs and Border Handling — Orientation and Evidence Baseline
- Customs and Border Handling — Ubiquitous Language, Aggregates, and State
- Customs and Border Handling — Commands, Events, APIs, Workflows, and Tools
- Customs and Border Handling — Data, Tables, Migrations, Lineage, and Reconciliation
- Customs and Border Handling — Portals, AI, Security, SLOs, Tests, and Operations
- Exceptions and Claims — Orientation and Evidence Baseline

Supplier — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-01-00`
Subject	`SupplierDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Supplier
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Supplier**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Supplier**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplierDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplierDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplierDomain`
- `ValidateSupplierDomain`
- `ApproveSupplierDomain`
- `ActivateSupplierDomain`
- `SuspendSupplierDomain`
- `CorrectSupplierDomain`
- `RetireSupplierDomain`

Reference Events:

- `SupplierDomainCreated`
- `SupplierDomainValidated`
- `SupplierDomainApproved`
- `SupplierDomainActivated`
- `SupplierDomainSuspended`
- `SupplierDomainCorrected`
- `SupplierDomainRetired`

Reference queries:

- `GetSupplierDomain`
- `ListSupplierDomainRecords`
- `GetSupplierDomainTimeline`
- `GetSupplierDomainEvidence`
- `GetSupplierDomainDecisionPackage`
- `GetSupplierDomainExceptions`
- `GetSupplierDomainAuditTrail`

Reference API surface:

```
POST /api/v1/supplierdomain/commands
GET  /api/v1/supplierdomain/{id}
GET  /api/v1/supplierdomain/{id}/timeline
GET  /api/v1/supplierdomain/{id}/evidence
GET  /api/v1/supplierdomain/{id}/decisions
POST /api/v1/supplierdomain/{id}/exceptions
POST /api/v1/supplierdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`supplierdomain_aggregate`	authoritative subject
`supplierdomain_version`	immutable submitted, approved, issued, or signed versions
`supplierdomain_relationship`	governed relationships
`supplierdomain_decision`	binding Decisions and rationale
`supplierdomain_evidence_link`	provenance and evidence relationships
`supplierdomain_event_outbox`	transactional Event publication
`supplierdomain_idempotency`	duplicate-write protection
`supplierdomain_exception`	exception, claim, dispute, or hold
`supplierdomain_correction`	correction and supersession lineage
`supplierdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Supplier` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Supplier — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-01-01`
Subject	`SupplierAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Supplier
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Supplier**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Supplier**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplierAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplierAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplierAggregateModel`
- `ValidateSupplierAggregateModel`
- `ApproveSupplierAggregateModel`
- `ActivateSupplierAggregateModel`
- `SuspendSupplierAggregateModel`
- `CorrectSupplierAggregateModel`
- `RetireSupplierAggregateModel`

Reference Events:

- `SupplierAggregateModelCreated`
- `SupplierAggregateModelValidated`
- `SupplierAggregateModelApproved`
- `SupplierAggregateModelActivated`
- `SupplierAggregateModelSuspended`
- `SupplierAggregateModelCorrected`
- `SupplierAggregateModelRetired`

Reference queries:

- `GetSupplierAggregateModel`
- `ListSupplierAggregateModelRecords`
- `GetSupplierAggregateModelTimeline`
- `GetSupplierAggregateModelEvidence`
- `GetSupplierAggregateModelDecisionPackage`
- `GetSupplierAggregateModelExceptions`
- `GetSupplierAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/supplieraggregatemodel/commands
GET  /api/v1/supplieraggregatemodel/{id}
GET  /api/v1/supplieraggregatemodel/{id}/timeline
GET  /api/v1/supplieraggregatemodel/{id}/evidence
GET  /api/v1/supplieraggregatemodel/{id}/decisions
POST /api/v1/supplieraggregatemodel/{id}/exceptions
POST /api/v1/supplieraggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`supplieraggregatemodel_aggregate`	authoritative subject
`supplieraggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`supplieraggregatemodel_relationship`	governed relationships
`supplieraggregatemodel_decision`	binding Decisions and rationale
`supplieraggregatemodel_evidence_link`	provenance and evidence relationships
`supplieraggregatemodel_event_outbox`	transactional Event publication
`supplieraggregatemodel_idempotency`	duplicate-write protection
`supplieraggregatemodel_exception`	exception, claim, dispute, or hold
`supplieraggregatemodel_correction`	correction and supersession lineage
`supplieraggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Supplier` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Supplier — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-01-02`
Subject	`SupplierContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Supplier
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Supplier**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Supplier**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplierContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplierContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplierContractModel`
- `ValidateSupplierContractModel`
- `ApproveSupplierContractModel`
- `ActivateSupplierContractModel`
- `SuspendSupplierContractModel`
- `CorrectSupplierContractModel`
- `RetireSupplierContractModel`

Reference Events:

- `SupplierContractModelCreated`
- `SupplierContractModelValidated`
- `SupplierContractModelApproved`
- `SupplierContractModelActivated`
- `SupplierContractModelSuspended`
- `SupplierContractModelCorrected`
- `SupplierContractModelRetired`

Reference queries:

- `GetSupplierContractModel`
- `ListSupplierContractModelRecords`
- `GetSupplierContractModelTimeline`
- `GetSupplierContractModelEvidence`
- `GetSupplierContractModelDecisionPackage`
- `GetSupplierContractModelExceptions`
- `GetSupplierContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/suppliercontractmodel/commands
GET  /api/v1/suppliercontractmodel/{id}
GET  /api/v1/suppliercontractmodel/{id}/timeline
GET  /api/v1/suppliercontractmodel/{id}/evidence
GET  /api/v1/suppliercontractmodel/{id}/decisions
POST /api/v1/suppliercontractmodel/{id}/exceptions
POST /api/v1/suppliercontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`suppliercontractmodel_aggregate`	authoritative subject
`suppliercontractmodel_version`	immutable submitted, approved, issued, or signed versions
`suppliercontractmodel_relationship`	governed relationships
`suppliercontractmodel_decision`	binding Decisions and rationale
`suppliercontractmodel_evidence_link`	provenance and evidence relationships
`suppliercontractmodel_event_outbox`	transactional Event publication
`suppliercontractmodel_idempotency`	duplicate-write protection
`suppliercontractmodel_exception`	exception, claim, dispute, or hold
`suppliercontractmodel_correction`	correction and supersession lineage
`suppliercontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Supplier` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Supplier — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-01-03`
Subject	`SupplierDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Supplier
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Supplier**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Supplier**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplierDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplierDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplierDataModel`
- `ValidateSupplierDataModel`
- `ApproveSupplierDataModel`
- `ActivateSupplierDataModel`
- `SuspendSupplierDataModel`
- `CorrectSupplierDataModel`
- `RetireSupplierDataModel`

Reference Events:

- `SupplierDataModelCreated`
- `SupplierDataModelValidated`
- `SupplierDataModelApproved`
- `SupplierDataModelActivated`
- `SupplierDataModelSuspended`
- `SupplierDataModelCorrected`
- `SupplierDataModelRetired`

Reference queries:

- `GetSupplierDataModel`
- `ListSupplierDataModelRecords`
- `GetSupplierDataModelTimeline`
- `GetSupplierDataModelEvidence`
- `GetSupplierDataModelDecisionPackage`
- `GetSupplierDataModelExceptions`
- `GetSupplierDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/supplierdatamodel/commands
GET  /api/v1/supplierdatamodel/{id}
GET  /api/v1/supplierdatamodel/{id}/timeline
GET  /api/v1/supplierdatamodel/{id}/evidence
GET  /api/v1/supplierdatamodel/{id}/decisions
POST /api/v1/supplierdatamodel/{id}/exceptions
POST /api/v1/supplierdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`supplierdatamodel_aggregate`	authoritative subject
`supplierdatamodel_version`	immutable submitted, approved, issued, or signed versions
`supplierdatamodel_relationship`	governed relationships
`supplierdatamodel_decision`	binding Decisions and rationale
`supplierdatamodel_evidence_link`	provenance and evidence relationships
`supplierdatamodel_event_outbox`	transactional Event publication
`supplierdatamodel_idempotency`	duplicate-write protection
`supplierdatamodel_exception`	exception, claim, dispute, or hold
`supplierdatamodel_correction`	correction and supersession lineage
`supplierdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Supplier` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Supplier — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-01-04`
Subject	`SupplierOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Supplier
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Supplier**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Supplier**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplierOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplierOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplierOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplierOperatingProfile`
- `ValidateSupplierOperatingProfile`
- `ApproveSupplierOperatingProfile`
- `ActivateSupplierOperatingProfile`
- `SuspendSupplierOperatingProfile`
- `CorrectSupplierOperatingProfile`
- `RetireSupplierOperatingProfile`

Reference Events:

- `SupplierOperatingProfileCreated`
- `SupplierOperatingProfileValidated`
- `SupplierOperatingProfileApproved`
- `SupplierOperatingProfileActivated`
- `SupplierOperatingProfileSuspended`
- `SupplierOperatingProfileCorrected`
- `SupplierOperatingProfileRetired`

Reference queries:

- `GetSupplierOperatingProfile`
- `ListSupplierOperatingProfileRecords`
- `GetSupplierOperatingProfileTimeline`
- `GetSupplierOperatingProfileEvidence`
- `GetSupplierOperatingProfileDecisionPackage`
- `GetSupplierOperatingProfileExceptions`
- `GetSupplierOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/supplieroperatingprofile/commands
GET  /api/v1/supplieroperatingprofile/{id}
GET  /api/v1/supplieroperatingprofile/{id}/timeline
GET  /api/v1/supplieroperatingprofile/{id}/evidence
GET  /api/v1/supplieroperatingprofile/{id}/decisions
POST /api/v1/supplieroperatingprofile/{id}/exceptions
POST /api/v1/supplieroperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`supplieroperatingprofile_aggregate`	authoritative subject
`supplieroperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`supplieroperatingprofile_relationship`	governed relationships
`supplieroperatingprofile_decision`	binding Decisions and rationale
`supplieroperatingprofile_evidence_link`	provenance and evidence relationships
`supplieroperatingprofile_event_outbox`	transactional Event publication
`supplieroperatingprofile_idempotency`	duplicate-write protection
`supplieroperatingprofile_exception`	exception, claim, dispute, or hold
`supplieroperatingprofile_correction`	correction and supersession lineage
`supplieroperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Supplier` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Provider — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-02-00`
Subject	`ProviderDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Provider
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Provider**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Provider**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProviderDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProviderDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProviderDomain`
- `ValidateProviderDomain`
- `ApproveProviderDomain`
- `ActivateProviderDomain`
- `SuspendProviderDomain`
- `CorrectProviderDomain`
- `RetireProviderDomain`

Reference Events:

- `ProviderDomainCreated`
- `ProviderDomainValidated`
- `ProviderDomainApproved`
- `ProviderDomainActivated`
- `ProviderDomainSuspended`
- `ProviderDomainCorrected`
- `ProviderDomainRetired`

Reference queries:

- `GetProviderDomain`
- `ListProviderDomainRecords`
- `GetProviderDomainTimeline`
- `GetProviderDomainEvidence`
- `GetProviderDomainDecisionPackage`
- `GetProviderDomainExceptions`
- `GetProviderDomainAuditTrail`

Reference API surface:

```
POST /api/v1/providerdomain/commands
GET  /api/v1/providerdomain/{id}
GET  /api/v1/providerdomain/{id}/timeline
GET  /api/v1/providerdomain/{id}/evidence
GET  /api/v1/providerdomain/{id}/decisions
POST /api/v1/providerdomain/{id}/exceptions
POST /api/v1/providerdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`providerdomain_aggregate`	authoritative subject
`providerdomain_version`	immutable submitted, approved, issued, or signed versions
`providerdomain_relationship`	governed relationships
`providerdomain_decision`	binding Decisions and rationale
`providerdomain_evidence_link`	provenance and evidence relationships
`providerdomain_event_outbox`	transactional Event publication
`providerdomain_idempotency`	duplicate-write protection
`providerdomain_exception`	exception, claim, dispute, or hold
`providerdomain_correction`	correction and supersession lineage
`providerdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Provider` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Provider — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-02-01`
Subject	`ProviderAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Provider
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Provider**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Provider**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProviderAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProviderAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProviderAggregateModel`
- `ValidateProviderAggregateModel`
- `ApproveProviderAggregateModel`
- `ActivateProviderAggregateModel`
- `SuspendProviderAggregateModel`
- `CorrectProviderAggregateModel`
- `RetireProviderAggregateModel`

Reference Events:

- `ProviderAggregateModelCreated`
- `ProviderAggregateModelValidated`
- `ProviderAggregateModelApproved`
- `ProviderAggregateModelActivated`
- `ProviderAggregateModelSuspended`
- `ProviderAggregateModelCorrected`
- `ProviderAggregateModelRetired`

Reference queries:

- `GetProviderAggregateModel`
- `ListProviderAggregateModelRecords`
- `GetProviderAggregateModelTimeline`
- `GetProviderAggregateModelEvidence`
- `GetProviderAggregateModelDecisionPackage`
- `GetProviderAggregateModelExceptions`
- `GetProviderAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/provideraggregatemodel/commands
GET  /api/v1/provideraggregatemodel/{id}
GET  /api/v1/provideraggregatemodel/{id}/timeline
GET  /api/v1/provideraggregatemodel/{id}/evidence
GET  /api/v1/provideraggregatemodel/{id}/decisions
POST /api/v1/provideraggregatemodel/{id}/exceptions
POST /api/v1/provideraggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`provideraggregatemodel_aggregate`	authoritative subject
`provideraggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`provideraggregatemodel_relationship`	governed relationships
`provideraggregatemodel_decision`	binding Decisions and rationale
`provideraggregatemodel_evidence_link`	provenance and evidence relationships
`provideraggregatemodel_event_outbox`	transactional Event publication
`provideraggregatemodel_idempotency`	duplicate-write protection
`provideraggregatemodel_exception`	exception, claim, dispute, or hold
`provideraggregatemodel_correction`	correction and supersession lineage
`provideraggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Provider` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Provider — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	`7`
Chapter ID	`V07-02-02`
Subject	`ProviderContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Provider
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Provider**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Provider**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProviderContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProviderContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProviderContractModel`
- `ValidateProviderContractModel`
- `ApproveProviderContractModel`
- `ActivateProviderContractModel`
- `SuspendProviderContractModel`
- `CorrectProviderContractModel`
- `RetireProviderContractModel`

Reference Events:

- `ProviderContractModelCreated`
- `ProviderContractModelValidated`
- `ProviderContractModelApproved`
- `ProviderContractModelActivated`
- `ProviderContractModelSuspended`
- `ProviderContractModelCorrected`
- `ProviderContractModelRetired`

Reference queries:

- `GetProviderContractModel`
- `ListProviderContractModelRecords`
- `GetProviderContractModelTimeline`
- `GetProviderContractModelEvidence`
- `GetProviderContractModelDecisionPackage`
- `GetProviderContractModelExceptions`
- `GetProviderContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/providercontractmodel/commands
GET  /api/v1/providercontractmodel/{id}
GET  /api/v1/providercontractmodel/{id}/timeline
GET  /api/v1/providercontractmodel/{id}/evidence
GET  /api/v1/providercontractmodel/{id}/decisions
POST /api/v1/providercontractmodel/{id}/exceptions
POST /api/v1/providercontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`providercontractmodel_aggregate`	authoritative subject
`providercontractmodel_version`	immutable submitted, approved, issued, or signed versions
`providercontractmodel_relationship`	governed relationships
`providercontractmodel_decision`	binding Decisions and rationale
`providercontractmodel_evidence_link`	provenance and evidence relationships
`providercontractmodel_event_outbox`	transactional Event publication
`providercontractmodel_idempotency`	duplicate-write protection
`providercontractmodel_exception`	exception, claim, dispute, or hold
`providercontractmodel_correction`	correction and supersession lineage
`providercontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Provider` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Provider — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-02-03`
Subject	`ProviderDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Provider
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Provider**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Provider**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProviderDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProviderDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProviderDataModel`
- `ValidateProviderDataModel`
- `ApproveProviderDataModel`
- `ActivateProviderDataModel`
- `SuspendProviderDataModel`
- `CorrectProviderDataModel`
- `RetireProviderDataModel`

Reference Events:

- `ProviderDataModelCreated`
- `ProviderDataModelValidated`
- `ProviderDataModelApproved`
- `ProviderDataModelActivated`
- `ProviderDataModelSuspended`
- `ProviderDataModelCorrected`
- `ProviderDataModelRetired`

Reference queries:

- `GetProviderDataModel`
- `ListProviderDataModelRecords`
- `GetProviderDataModelTimeline`
- `GetProviderDataModelEvidence`
- `GetProviderDataModelDecisionPackage`
- `GetProviderDataModelExceptions`
- `GetProviderDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/providerdatamodel/commands
GET  /api/v1/providerdatamodel/{id}
GET  /api/v1/providerdatamodel/{id}/timeline
GET  /api/v1/providerdatamodel/{id}/evidence
GET  /api/v1/providerdatamodel/{id}/decisions
POST /api/v1/providerdatamodel/{id}/exceptions
POST /api/v1/providerdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`providerdatamodel_aggregate`	authoritative subject
`providerdatamodel_version`	immutable submitted, approved, issued, or signed versions
`providerdatamodel_relationship`	governed relationships
`providerdatamodel_decision`	binding Decisions and rationale
`providerdatamodel_evidence_link`	provenance and evidence relationships
`providerdatamodel_event_outbox`	transactional Event publication
`providerdatamodel_idempotency`	duplicate-write protection
`providerdatamodel_exception`	exception, claim, dispute, or hold
`providerdatamodel_correction`	correction and supersession lineage
`providerdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Provider` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Provider — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-02-04`
Subject	`ProviderOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Provider
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Provider**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Provider**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProviderOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProviderOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProviderOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProviderOperatingProfile`
- `ValidateProviderOperatingProfile`
- `ApproveProviderOperatingProfile`
- `ActivateProviderOperatingProfile`
- `SuspendProviderOperatingProfile`
- `CorrectProviderOperatingProfile`
- `RetireProviderOperatingProfile`

Reference Events:

- `ProviderOperatingProfileCreated`
- `ProviderOperatingProfileValidated`
- `ProviderOperatingProfileApproved`
- `ProviderOperatingProfileActivated`
- `ProviderOperatingProfileSuspended`
- `ProviderOperatingProfileCorrected`
- `ProviderOperatingProfileRetired`

Reference queries:

- `GetProviderOperatingProfile`
- `ListProviderOperatingProfileRecords`
- `GetProviderOperatingProfileTimeline`
- `GetProviderOperatingProfileEvidence`
- `GetProviderOperatingProfileDecisionPackage`
- `GetProviderOperatingProfileExceptions`
- `GetProviderOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/provideroperatingprofile/commands
GET  /api/v1/provideroperatingprofile/{id}
GET  /api/v1/provideroperatingprofile/{id}/timeline
GET  /api/v1/provideroperatingprofile/{id}/evidence
GET  /api/v1/provideroperatingprofile/{id}/decisions
POST /api/v1/provideroperatingprofile/{id}/exceptions
POST /api/v1/provideroperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`provideroperatingprofile_aggregate`	authoritative subject
`provideroperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`provideroperatingprofile_relationship`	governed relationships
`provideroperatingprofile_decision`	binding Decisions and rationale
`provideroperatingprofile_evidence_link`	provenance and evidence relationships
`provideroperatingprofile_event_outbox`	transactional Event publication
`provideroperatingprofile_idempotency`	duplicate-write protection
`provideroperatingprofile_exception`	exception, claim, dispute, or hold
`provideroperatingprofile_correction`	correction and supersession lineage
`provideroperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Provider` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter’s evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customer — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-03-00`
Subject	`CustomerDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customer
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Customer**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customer**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomerDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomerDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomerDomain`
- `ValidateCustomerDomain`
- `ApproveCustomerDomain`
- `ActivateCustomerDomain`
- `SuspendCustomerDomain`
- `CorrectCustomerDomain`
- `RetireCustomerDomain`

Reference Events:

- `CustomerDomainCreated`
- `CustomerDomainValidated`
- `CustomerDomainApproved`
- `CustomerDomainActivated`
- `CustomerDomainSuspended`
- `CustomerDomainCorrected`
- `CustomerDomainRetired`

Reference queries:

- `GetCustomerDomain`
- `ListCustomerDomainRecords`
- `GetCustomerDomainTimeline`
- `GetCustomerDomainEvidence`
- `GetCustomerDomainDecisionPackage`
- `GetCustomerDomainExceptions`
- `GetCustomerDomainAuditTrail`

Reference API surface:

```
POST /api/v1/customerdomain/commands
GET  /api/v1/customerdomain/{id}
GET  /api/v1/customerdomain/{id}/timeline
GET  /api/v1/customerdomain/{id}/evidence
GET  /api/v1/customerdomain/{id}/decisions
POST /api/v1/customerdomain/{id}/exceptions
POST /api/v1/customerdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customerdomain_aggregate`	authoritative subject
`customerdomain_version`	immutable submitted, approved, issued, or signed versions
`customerdomain_relationship`	governed relationships
`customerdomain_decision`	binding Decisions and rationale
`customerdomain_evidence_link`	provenance and evidence relationships
`customerdomain_event_outbox`	transactional Event publication
`customerdomain_idempotency`	duplicate-write protection
`customerdomain_exception`	exception, claim, dispute, or hold
`customerdomain_correction`	correction and supersession lineage
`customerdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customer` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Customer — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-03-01`
Subject	`CustomerAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customer
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Customer**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customer**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomerAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomerAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomerAggregateModel`
- `ValidateCustomerAggregateModel`
- `ApproveCustomerAggregateModel`
- `ActivateCustomerAggregateModel`
- `SuspendCustomerAggregateModel`
- `CorrectCustomerAggregateModel`
- `RetireCustomerAggregateModel`

Reference Events:

- `CustomerAggregateModelCreated`
- `CustomerAggregateModelValidated`
- `CustomerAggregateModelApproved`
- `CustomerAggregateModelActivated`
- `CustomerAggregateModelSuspended`
- `CustomerAggregateModelCorrected`
- `CustomerAggregateModelRetired`

Reference queries:

- `GetCustomerAggregateModel`
- `ListCustomerAggregateModelRecords`
- `GetCustomerAggregateModelTimeline`
- `GetCustomerAggregateModelEvidence`
- `GetCustomerAggregateModelDecisionPackage`
- `GetCustomerAggregateModelExceptions`
- `GetCustomerAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/customeraggregatemodel/commands
GET  /api/v1/customeraggregatemodel/{id}
GET  /api/v1/customeraggregatemodel/{id}/timeline
GET  /api/v1/customeraggregatemodel/{id}/evidence
GET  /api/v1/customeraggregatemodel/{id}/decisions
POST /api/v1/customeraggregatemodel/{id}/exceptions
POST /api/v1/customeraggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customeraggregatemodel_aggregate`	authoritative subject
`customeraggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`customeraggregatemodel_relationship`	governed relationships
`customeraggregatemodel_decision`	binding Decisions and rationale
`customeraggregatemodel_evidence_link`	provenance and evidence relationships
`customeraggregatemodel_event_outbox`	transactional Event publication
`customeraggregatemodel_idempotency`	duplicate-write protection
`customeraggregatemodel_exception`	exception, claim, dispute, or hold
`customeraggregatemodel_correction`	correction and supersession lineage
`customeraggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customer` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customer — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-03-02`
Subject	`CustomerContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customer
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Customer**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customer**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomerContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomerContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomerContractModel`
- `ValidateCustomerContractModel`
- `ApproveCustomerContractModel`
- `ActivateCustomerContractModel`
- `SuspendCustomerContractModel`
- `CorrectCustomerContractModel`
- `RetireCustomerContractModel`

Reference Events:

- `CustomerContractModelCreated`
- `CustomerContractModelValidated`
- `CustomerContractModelApproved`
- `CustomerContractModelActivated`
- `CustomerContractModelSuspended`
- `CustomerContractModelCorrected`
- `CustomerContractModelRetired`

Reference queries:

- `GetCustomerContractModel`
- `ListCustomerContractModelRecords`
- `GetCustomerContractModelTimeline`
- `GetCustomerContractModelEvidence`
- `GetCustomerContractModelDecisionPackage`
- `GetCustomerContractModelExceptions`
- `GetCustomerContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/customercontractmodel/commands
GET  /api/v1/customercontractmodel/{id}
GET  /api/v1/customercontractmodel/{id}/timeline
GET  /api/v1/customercontractmodel/{id}/evidence
GET  /api/v1/customercontractmodel/{id}/decisions
POST /api/v1/customercontractmodel/{id}/exceptions
POST /api/v1/customercontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customercontractmodel_aggregate`	authoritative subject
`customercontractmodel_version`	immutable submitted, approved, issued, or signed versions
`customercontractmodel_relationship`	governed relationships
`customercontractmodel_decision`	binding Decisions and rationale
`customercontractmodel_evidence_link`	provenance and evidence relationships
`customercontractmodel_event_outbox`	transactional Event publication
`customercontractmodel_idempotency`	duplicate-write protection
`customercontractmodel_exception`	exception, claim, dispute, or hold
`customercontractmodel_correction`	correction and supersession lineage
`customercontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customer` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customer — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-03-03`
Subject	`CustomerDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customer
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Customer**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customer**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomerDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomerDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomerDataModel`
- `ValidateCustomerDataModel`
- `ApproveCustomerDataModel`
- `ActivateCustomerDataModel`
- `SuspendCustomerDataModel`
- `CorrectCustomerDataModel`
- `RetireCustomerDataModel`

Reference Events:

- `CustomerDataModelCreated`
- `CustomerDataModelValidated`
- `CustomerDataModelApproved`
- `CustomerDataModelActivated`
- `CustomerDataModelSuspended`
- `CustomerDataModelCorrected`
- `CustomerDataModelRetired`

Reference queries:

- `GetCustomerDataModel`
- `ListCustomerDataModelRecords`
- `GetCustomerDataModelTimeline`
- `GetCustomerDataModelEvidence`
- `GetCustomerDataModelDecisionPackage`
- `GetCustomerDataModelExceptions`
- `GetCustomerDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/customerdatamodel/commands
GET  /api/v1/customerdatamodel/{id}
GET  /api/v1/customerdatamodel/{id}/timeline
GET  /api/v1/customerdatamodel/{id}/evidence
GET  /api/v1/customerdatamodel/{id}/decisions
POST /api/v1/customerdatamodel/{id}/exceptions
POST /api/v1/customerdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customerdatamodel_aggregate`	authoritative subject
`customerdatamodel_version`	immutable submitted, approved, issued, or signed versions
`customerdatamodel_relationship`	governed relationships
`customerdatamodel_decision`	binding Decisions and rationale
`customerdatamodel_evidence_link`	provenance and evidence relationships
`customerdatamodel_event_outbox`	transactional Event publication
`customerdatamodel_idempotency`	duplicate-write protection
`customerdatamodel_exception`	exception, claim, dispute, or hold
`customerdatamodel_correction`	correction and supersession lineage
`customerdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customer` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customer — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-03-04`
Subject	`CustomerOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customer
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Customer**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customer**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomerOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomerOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomerOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomerOperatingProfile`
- `ValidateCustomerOperatingProfile`
- `ApproveCustomerOperatingProfile`
- `ActivateCustomerOperatingProfile`
- `SuspendCustomerOperatingProfile`
- `CorrectCustomerOperatingProfile`
- `RetireCustomerOperatingProfile`

Reference Events:

- `CustomerOperatingProfileCreated`
- `CustomerOperatingProfileValidated`
- `CustomerOperatingProfileApproved`
- `CustomerOperatingProfileActivated`
- `CustomerOperatingProfileSuspended`
- `CustomerOperatingProfileCorrected`
- `CustomerOperatingProfileRetired`

Reference queries:

- `GetCustomerOperatingProfile`
- `ListCustomerOperatingProfileRecords`
- `GetCustomerOperatingProfileTimeline`
- `GetCustomerOperatingProfileEvidence`
- `GetCustomerOperatingProfileDecisionPackage`
- `GetCustomerOperatingProfileExceptions`
- `GetCustomerOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/customeroperatingprofile/commands
GET  /api/v1/customeroperatingprofile/{id}
GET  /api/v1/customeroperatingprofile/{id}/timeline
GET  /api/v1/customeroperatingprofile/{id}/evidence
GET  /api/v1/customeroperatingprofile/{id}/decisions
POST /api/v1/customeroperatingprofile/{id}/exceptions
POST /api/v1/customeroperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customeroperatingprofile_aggregate`	authoritative subject
`customeroperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`customeroperatingprofile_relationship`	governed relationships
`customeroperatingprofile_decision`	binding Decisions and rationale
`customeroperatingprofile_evidence_link`	provenance and evidence relationships
`customeroperatingprofile_event_outbox`	transactional Event publication
`customeroperatingprofile_idempotency`	duplicate-write protection
`customeroperatingprofile_exception`	exception, claim, dispute, or hold
`customeroperatingprofile_correction`	correction and supersession lineage
`customeroperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customer` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

CRM — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	V07-04-00
Subject	CRMDomain
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / CRM
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of CRM**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / CRM**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CRMDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CRMDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCRMDomain`
- `ValidateCRMDomain`
- `ApproveCRMDomain`
- `ActivateCRMDomain`
- `SuspendCRMDomain`
- `CorrectCRMDomain`
- `RetireCRMDomain`

Reference Events:

- `CRMDomainCreated`
- `CRMDomainValidated`
- `CRMDomainApproved`
- `CRMDomainActivated`
- `CRMDomainSuspended`
- `CRMDomainCorrected`
- `CRMDomainRetired`

Reference queries:

- `GetCRMDomain`
- `ListCRMDomainRecords`
- `GetCRMDomainTimeline`
- `GetCRMDomainEvidence`
- `GetCRMDomainDecisionPackage`
- `GetCRMDomainExceptions`
- `GetCRMDomainAuditTrail`

Reference API surface:

```
POST /api/v1/crmdomain/commands
GET  /api/v1/crmdomain/{id}
GET  /api/v1/crmdomain/{id}/timeline
GET  /api/v1/crmdomain/{id}/evidence
GET  /api/v1/crmdomain/{id}/decisions
POST /api/v1/crmdomain/{id}/exceptions
POST /api/v1/crmdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`crmdomain_aggregate`	authoritative subject
`crmdomain_version`	immutable submitted, approved, issued, or signed versions
`crmdomain_relationship`	governed relationships
`crmdomain_decision`	binding Decisions and rationale
`crmdomain_evidence_link`	provenance and evidence relationships
`crmdomain_event_outbox`	transactional Event publication
`crmdomain_idempotency`	duplicate-write protection
`crmdomain_exception`	exception, claim, dispute, or hold
`crmdomain_correction`	correction and supersession lineage
`crmdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `CRM` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

CRM — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-04-01`
Subject	`CRMAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / CRM
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, life-cycle state, effective time, correction, and concurrency for CRM**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / CRM**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CRMAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CRMAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCRMAggregateModel`
- `ValidateCRMAggregateModel`
- `ApproveCRMAggregateModel`
- `ActivateCRMAggregateModel`
- `SuspendCRMAggregateModel`
- `CorrectCRMAggregateModel`
- `RetireCRMAggregateModel`

Reference Events:

- `CRMAggregateModelCreated`
- `CRMAggregateModelValidated`
- `CRMAggregateModelApproved`
- `CRMAggregateModelActivated`
- `CRMAggregateModelSuspended`
- `CRMAggregateModelCorrected`
- `CRMAggregateModelRetired`

Reference queries:

- `GetCRMAggregateModel`
- `ListCRMAggregateModelRecords`
- `GetCRMAggregateModelTimeline`
- `GetCRMAggregateModelEvidence`
- `GetCRMAggregateModelDecisionPackage`
- `GetCRMAggregateModelExceptions`
- `GetCRMAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/crmaggregatemodel/commands
GET  /api/v1/crmaggregatemodel/{id}
GET  /api/v1/crmaggregatemodel/{id}/timeline
GET  /api/v1/crmaggregatemodel/{id}/evidence
GET  /api/v1/crmaggregatemodel/{id}/decisions
POST /api/v1/crmaggregatemodel/{id}/exceptions
POST /api/v1/crmaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`crmaggregatemodel_aggregate`	authoritative subject
`crmaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`crmaggregatemodel_relationship`	governed relationships
`crmaggregatemodel_decision`	binding Decisions and rationale
`crmaggregatemodel_evidence_link`	provenance and evidence relationships
`crmaggregatemodel_event_outbox`	transactional Event publication
`crmaggregatemodel_idempotency`	duplicate-write protection
`crmaggregatemodel_exception`	exception, claim, dispute, or hold
`crmaggregatemodel_correction`	correction and supersession lineage
`crmaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `CRM` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

CRM — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	V07-04-02
Subject	CRMContractModel
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / CRM
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for CRM**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / CRM**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CRMContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CRMContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCRMContractModel`
- `ValidateCRMContractModel`
- `ApproveCRMContractModel`
- `ActivateCRMContractModel`
- `SuspendCRMContractModel`
- `CorrectCRMContractModel`
- `RetireCRMContractModel`

Reference Events:

- `CRMContractModelCreated`
- `CRMContractModelValidated`
- `CRMContractModelApproved`
- `CRMContractModelActivated`
- `CRMContractModelSuspended`
- `CRMContractModelCorrected`
- `CRMContractModelRetired`

Reference queries:

- `GetCRMContractModel`
- `ListCRMContractModelRecords`
- `GetCRMContractModelTimeline`
- `GetCRMContractModelEvidence`
- `GetCRMContractModelDecisionPackage`
- `GetCRMContractModelExceptions`
- `GetCRMContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/crmcontractmodel/commands
GET  /api/v1/crmcontractmodel/{id}
GET  /api/v1/crmcontractmodel/{id}/timeline
GET  /api/v1/crmcontractmodel/{id}/evidence
GET  /api/v1/crmcontractmodel/{id}/decisions
POST /api/v1/crmcontractmodel/{id}/exceptions
POST /api/v1/crmcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`crmcontractmodel_aggregate`	authoritative subject
`crmcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`crmcontractmodel_relationship`	governed relationships
`crmcontractmodel_decision`	binding Decisions and rationale
`crmcontractmodel_evidence_link`	provenance and evidence relationships
`crmcontractmodel_event_outbox`	transactional Event publication
`crmcontractmodel_idempotency`	duplicate-write protection
`crmcontractmodel_exception`	exception, claim, dispute, or hold
`crmcontractmodel_correction`	correction and supersession lineage
`crmcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `CRM` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

CRM — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-04-03`
Subject	`CRMDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / CRM
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for CRM**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / CRM**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CRMDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CRMDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCRMDataModel`
- `ValidateCRMDataModel`
- `ApproveCRMDataModel`
- `ActivateCRMDataModel`
- `SuspendCRMDataModel`
- `CorrectCRMDataModel`
- `RetireCRMDataModel`

Reference Events:

- `CRMDataModelCreated`
- `CRMDataModelValidated`
- `CRMDataModelApproved`
- `CRMDataModelActivated`
- `CRMDataModelSuspended`
- `CRMDataModelCorrected`
- `CRMDataModelRetired`

Reference queries:

- `GetCRMDataModel`
- `ListCRMDataModelRecords`
- `GetCRMDataModelTimeline`
- `GetCRMDataModelEvidence`
- `GetCRMDataModelDecisionPackage`
- `GetCRMDataModelExceptions`
- `GetCRMDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/crmdatamodel/commands
GET  /api/v1/crmdatamodel/{id}
GET  /api/v1/crmdatamodel/{id}/timeline
GET  /api/v1/crmdatamodel/{id}/evidence
GET  /api/v1/crmdatamodel/{id}/decisions
POST /api/v1/crmdatamodel/{id}/exceptions
POST /api/v1/crmdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`crmdatamodel_aggregate`	authoritative subject
`crmdatamodel_version`	immutable submitted, approved, issued, or signed versions
`crmdatamodel_relationship`	governed relationships
`crmdatamodel_decision`	binding Decisions and rationale
`crmdatamodel_evidence_link`	provenance and evidence relationships
`crmdatamodel_event_outbox`	transactional Event publication
`crmdatamodel_idempotency`	duplicate-write protection
`crmdatamodel_exception`	exception, claim, dispute, or hold
`crmdatamodel_correction`	correction and supersession lineage
`crmdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `CRM` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

CRM — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-04-04`
Subject	`CRMOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / CRM
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for CRM**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / CRM**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CRMOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CRMOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CRMOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCRMOperatingProfile`
- `ValidateCRMOperatingProfile`
- `ApproveCRMOperatingProfile`
- `ActivateCRMOperatingProfile`
- `SuspendCRMOperatingProfile`
- `CorrectCRMOperatingProfile`
- `RetireCRMOperatingProfile`

Reference Events:

- `CRMOperatingProfileCreated`
- `CRMOperatingProfileValidated`
- `CRMOperatingProfileApproved`
- `CRMOperatingProfileActivated`
- `CRMOperatingProfileSuspended`
- `CRMOperatingProfileCorrected`
- `CRMOperatingProfileRetired`

Reference queries:

- `GetCRMOperatingProfile`
- `ListCRMOperatingProfileRecords`
- `GetCRMOperatingProfileTimeline`
- `GetCRMOperatingProfileEvidence`
- `GetCRMOperatingProfileDecisionPackage`
- `GetCRMOperatingProfileExceptions`
- `GetCRMOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/crmoperatingprofile/commands
GET  /api/v1/crmoperatingprofile/{id}
GET  /api/v1/crmoperatingprofile/{id}/timeline
GET  /api/v1/crmoperatingprofile/{id}/evidence
GET  /api/v1/crmoperatingprofile/{id}/decisions
POST /api/v1/crmoperatingprofile/{id}/exceptions
POST /api/v1/crmoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`crmoperatingprofile_aggregate`	authoritative subject
`crmoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`crmoperatingprofile_relationship`	governed relationships
`crmoperatingprofile_decision`	binding Decisions and rationale
`crmoperatingprofile_evidence_link`	provenance and evidence relationships
`crmoperatingprofile_event_outbox`	transactional Event publication
`crmoperatingprofile_idempotency`	duplicate-write protection
`crmoperatingprofile_exception`	exception, claim, dispute, or hold
`crmoperatingprofile_correction`	correction and supersession lineage
`crmoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `CRM` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Inventory — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-05-00`
Subject	`InventoryDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Inventory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Inventory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Inventory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`InventoryDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
InventoryDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateInventoryDomain`
- `ValidateInventoryDomain`
- `ApproveInventoryDomain`
- `ActivateInventoryDomain`
- `SuspendInventoryDomain`
- `CorrectInventoryDomain`
- `RetireInventoryDomain`

Reference Events:

- `InventoryDomainCreated`
- `InventoryDomainValidated`
- `InventoryDomainApproved`
- `InventoryDomainActivated`
- `InventoryDomainSuspended`
- `InventoryDomainCorrected`
- `InventoryDomainRetired`

Reference queries:

- `GetInventoryDomain`
- `ListInventoryDomainRecords`
- `GetInventoryDomainTimeline`
- `GetInventoryDomainEvidence`
- `GetInventoryDomainDecisionPackage`
- `GetInventoryDomainExceptions`
- `GetInventoryDomainAuditTrail`

Reference API surface:

```
POST /api/v1/inventorydomain/commands
GET  /api/v1/inventorydomain/{id}
GET  /api/v1/inventorydomain/{id}/timeline
GET  /api/v1/inventorydomain/{id}/evidence
GET  /api/v1/inventorydomain/{id}/decisions
POST /api/v1/inventorydomain/{id}/exceptions
POST /api/v1/inventorydomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`inventorydomain_aggregate`	authoritative subject
`inventorydomain_version`	immutable submitted, approved, issued, or signed versions
`inventorydomain_relationship`	governed relationships
`inventorydomain_decision`	binding Decisions and rationale
`inventorydomain_evidence_link`	provenance and evidence relationships
`inventorydomain_event_outbox`	transactional Event publication
`inventorydomain_idempotency`	duplicate-write protection
`inventorydomain_exception`	exception, claim, dispute, or hold
`inventorydomain_correction`	correction and supersession lineage
`inventorydomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Inventory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Inventory — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-05-01`
Subject	`InventoryAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Inventory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, life-cycle state, effective time, correction, and concurrency for Inventory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Inventory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`InventoryAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
InventoryAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateInventoryAggregateModel`
- `ValidateInventoryAggregateModel`
- `ApproveInventoryAggregateModel`
- `ActivateInventoryAggregateModel`
- `SuspendInventoryAggregateModel`
- `CorrectInventoryAggregateModel`
- `RetireInventoryAggregateModel`

Reference Events:

- `InventoryAggregateModelCreated`
- `InventoryAggregateModelValidated`
- `InventoryAggregateModelApproved`
- `InventoryAggregateModelActivated`
- `InventoryAggregateModelSuspended`
- `InventoryAggregateModelCorrected`
- `InventoryAggregateModelRetired`

Reference queries:

- `GetInventoryAggregateModel`
- `ListInventoryAggregateModelRecords`
- `GetInventoryAggregateModelTimeline`
- `GetInventoryAggregateModelEvidence`
- `GetInventoryAggregateModelDecisionPackage`
- `GetInventoryAggregateModelExceptions`
- `GetInventoryAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/inventoryaggregatemodel/commands
GET  /api/v1/inventoryaggregatemodel/{id}
GET  /api/v1/inventoryaggregatemodel/{id}/timeline
GET  /api/v1/inventoryaggregatemodel/{id}/evidence
GET  /api/v1/inventoryaggregatemodel/{id}/decisions
POST /api/v1/inventoryaggregatemodel/{id}/exceptions
POST /api/v1/inventoryaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`inventoryaggregatemodel_aggregate`	authoritative subject
`inventoryaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`inventoryaggregatemodel_relationship`	governed relationships
`inventoryaggregatemodel_decision`	binding Decisions and rationale
`inventoryaggregatemodel_evidence_link`	provenance and evidence relationships
`inventoryaggregatemodel_event_outbox`	transactional Event publication
`inventoryaggregatemodel_idempotency`	duplicate-write protection
`inventoryaggregatemodel_exception`	exception, claim, dispute, or hold
`inventoryaggregatemodel_correction`	correction and supersession lineage
`inventoryaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Inventory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Inventory — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-05-02`
Subject	`InventoryContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Inventory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Inventory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Inventory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`InventoryContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
InventoryContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateInventoryContractModel`
- `ValidateInventoryContractModel`
- `ApproveInventoryContractModel`
- `ActivateInventoryContractModel`
- `SuspendInventoryContractModel`
- `CorrectInventoryContractModel`
- `RetireInventoryContractModel`

Reference Events:

- `InventoryContractModelCreated`
- `InventoryContractModelValidated`
- `InventoryContractModelApproved`
- `InventoryContractModelActivated`
- `InventoryContractModelSuspended`
- `InventoryContractModelCorrected`
- `InventoryContractModelRetired`

Reference queries:

- `GetInventoryContractModel`
- `ListInventoryContractModelRecords`
- `GetInventoryContractModelTimeline`
- `GetInventoryContractModelEvidence`
- `GetInventoryContractModelDecisionPackage`
- `GetInventoryContractModelExceptions`
- `GetInventoryContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/inventorycontractmodel/commands
GET  /api/v1/inventorycontractmodel/{id}
GET  /api/v1/inventorycontractmodel/{id}/timeline
GET  /api/v1/inventorycontractmodel/{id}/evidence
GET  /api/v1/inventorycontractmodel/{id}/decisions
POST /api/v1/inventorycontractmodel/{id}/exceptions
POST /api/v1/inventorycontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`inventorycontractmodel_aggregate`	authoritative subject
`inventorycontractmodel_version`	immutable submitted, approved, issued, or signed versions
`inventorycontractmodel_relationship`	governed relationships
`inventorycontractmodel_decision`	binding Decisions and rationale
`inventorycontractmodel_evidence_link`	provenance and evidence relationships
`inventorycontractmodel_event_outbox`	transactional Event publication
`inventorycontractmodel_idempotency`	duplicate-write protection
`inventorycontractmodel_exception`	exception, claim, dispute, or hold
`inventorycontractmodel_correction`	correction and supersession lineage
`inventorycontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Inventory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Inventory — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-05-03`
Subject	`InventoryDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Inventory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Inventory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Inventory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`InventoryDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
InventoryDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateInventoryDataModel`
- `ValidateInventoryDataModel`
- `ApproveInventoryDataModel`
- `ActivateInventoryDataModel`
- `SuspendInventoryDataModel`
- `CorrectInventoryDataModel`
- `RetireInventoryDataModel`

Reference Events:

- `InventoryDataModelCreated`
- `InventoryDataModelValidated`
- `InventoryDataModelApproved`
- `InventoryDataModelActivated`
- `InventoryDataModelSuspended`
- `InventoryDataModelCorrected`
- `InventoryDataModelRetired`

Reference queries:

- `GetInventoryDataModel`
- `ListInventoryDataModelRecords`
- `GetInventoryDataModelTimeline`
- `GetInventoryDataModelEvidence`
- `GetInventoryDataModelDecisionPackage`
- `GetInventoryDataModelExceptions`
- `GetInventoryDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/inventorydatamodel/commands
GET  /api/v1/inventorydatamodel/{id}
GET  /api/v1/inventorydatamodel/{id}/timeline
GET  /api/v1/inventorydatamodel/{id}/evidence
GET  /api/v1/inventorydatamodel/{id}/decisions
POST /api/v1/inventorydatamodel/{id}/exceptions
POST /api/v1/inventorydatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`inventorydatamodel_aggregate`	authoritative subject
`inventorydatamodel_version`	immutable submitted, approved, issued, or signed versions
`inventorydatamodel_relationship`	governed relationships
`inventorydatamodel_decision`	binding Decisions and rationale
`inventorydatamodel_evidence_link`	provenance and evidence relationships
`inventorydatamodel_event_outbox`	transactional Event publication
`inventorydatamodel_idempotency`	duplicate-write protection
`inventorydatamodel_exception`	exception, claim, dispute, or hold
`inventorydatamodel_correction`	correction and supersession lineage
`inventorydatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Inventory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Inventory — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-05-04`
Subject	`InventoryOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Inventory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Inventory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Inventory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`InventoryOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`InventoryOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
InventoryOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateInventoryOperatingProfile`
- `ValidateInventoryOperatingProfile`
- `ApproveInventoryOperatingProfile`
- `ActivateInventoryOperatingProfile`
- `SuspendInventoryOperatingProfile`
- `CorrectInventoryOperatingProfile`
- `RetireInventoryOperatingProfile`

Reference Events:

- `InventoryOperatingProfileCreated`
- `InventoryOperatingProfileValidated`
- `InventoryOperatingProfileApproved`
- `InventoryOperatingProfileActivated`
- `InventoryOperatingProfileSuspended`
- `InventoryOperatingProfileCorrected`
- `InventoryOperatingProfileRetired`

Reference queries:

- `GetInventoryOperatingProfile`
- `ListInventoryOperatingProfileRecords`
- `GetInventoryOperatingProfileTimeline`
- `GetInventoryOperatingProfileEvidence`
- `GetInventoryOperatingProfileDecisionPackage`
- `GetInventoryOperatingProfileExceptions`
- `GetInventoryOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/inventoryoperatingprofile/commands
GET  /api/v1/inventoryoperatingprofile/{id}
GET  /api/v1/inventoryoperatingprofile/{id}/timeline
GET  /api/v1/inventoryoperatingprofile/{id}/evidence
GET  /api/v1/inventoryoperatingprofile/{id}/decisions
POST /api/v1/inventoryoperatingprofile/{id}/exceptions
POST /api/v1/inventoryoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`inventoryoperatingprofile_aggregate`	authoritative subject
`inventoryoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`inventoryoperatingprofile_relationship`	governed relationships
`inventoryoperatingprofile_decision`	binding Decisions and rationale
`inventoryoperatingprofile_evidence_link`	provenance and evidence relationships
`inventoryoperatingprofile_event_outbox`	transactional Event publication
`inventoryoperatingprofile_idempotency`	duplicate-write protection
`inventoryoperatingprofile_exception`	exception, claim, dispute, or hold
`inventoryoperatingprofile_correction`	correction and supersession lineage
`inventoryoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Inventory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter’s evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Warehouse — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	V07-06-00
Subject	WarehouseDomain
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / Warehouse
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Warehouse**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Warehouse**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WarehouseDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WarehouseDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWarehouseDomain`
- `ValidateWarehouseDomain`
- `ApproveWarehouseDomain`
- `ActivateWarehouseDomain`
- `SuspendWarehouseDomain`
- `CorrectWarehouseDomain`
- `RetireWarehouseDomain`

Reference Events:

- `WarehouseDomainCreated`
- `WarehouseDomainValidated`
- `WarehouseDomainApproved`
- `WarehouseDomainActivated`
- `WarehouseDomainSuspended`
- `WarehouseDomainCorrected`
- `WarehouseDomainRetired`

Reference queries:

- `GetWarehouseDomain`
- `ListWarehouseDomainRecords`
- `GetWarehouseDomainTimeline`
- `GetWarehouseDomainEvidence`
- `GetWarehouseDomainDecisionPackage`
- `GetWarehouseDomainExceptions`
- `GetWarehouseDomainAuditTrail`

Reference API surface:

```
POST /api/v1/warehousedomain/commands
GET  /api/v1/warehousedomain/{id}
GET  /api/v1/warehousedomain/{id}/timeline
GET  /api/v1/warehousedomain/{id}/evidence
GET  /api/v1/warehousedomain/{id}/decisions
POST /api/v1/warehousedomain/{id}/exceptions
POST /api/v1/warehousedomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`warehousedomain_aggregate`	authoritative subject
`warehousedomain_version`	immutable submitted, approved, issued, or signed versions
`warehousedomain_relationship`	governed relationships
`warehousedomain_decision`	binding Decisions and rationale
`warehousedomain_evidence_link`	provenance and evidence relationships
`warehousedomain_event_outbox`	transactional Event publication
`warehousedomain_idempotency`	duplicate-write protection
`warehousedomain_exception`	exception, claim, dispute, or hold
`warehousedomain_correction`	correction and supersession lineage
`warehousedomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Warehouse` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Warehouse — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-06-01`
Subject	`WarehouseAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Warehouse
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Warehouse**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Warehouse**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WarehouseAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WarehouseAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWarehouseAggregateModel`
- `ValidateWarehouseAggregateModel`
- `ApproveWarehouseAggregateModel`
- `ActivateWarehouseAggregateModel`
- `SuspendWarehouseAggregateModel`
- `CorrectWarehouseAggregateModel`
- `RetireWarehouseAggregateModel`

Reference Events:

- `WarehouseAggregateModelCreated`
- `WarehouseAggregateModelValidated`
- `WarehouseAggregateModelApproved`
- `WarehouseAggregateModelActivated`
- `WarehouseAggregateModelSuspended`
- `WarehouseAggregateModelCorrected`
- `WarehouseAggregateModelRetired`

Reference queries:

- `GetWarehouseAggregateModel`
- `ListWarehouseAggregateModelRecords`
- `GetWarehouseAggregateModelTimeline`
- `GetWarehouseAggregateModelEvidence`
- `GetWarehouseAggregateModelDecisionPackage`
- `GetWarehouseAggregateModelExceptions`
- `GetWarehouseAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/warehouseaggregatemodel/commands
GET  /api/v1/warehouseaggregatemodel/{id}
GET  /api/v1/warehouseaggregatemodel/{id}/timeline
GET  /api/v1/warehouseaggregatemodel/{id}/evidence
GET  /api/v1/warehouseaggregatemodel/{id}/decisions
POST /api/v1/warehouseaggregatemodel/{id}/exceptions
POST /api/v1/warehouseaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`warehouseaggregatemodel_aggregate`	authoritative subject
`warehouseaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`warehouseaggregatemodel_relationship`	governed relationships
`warehouseaggregatemodel_decision`	binding Decisions and rationale
`warehouseaggregatemodel_evidence_link`	provenance and evidence relationships
`warehouseaggregatemodel_event_outbox`	transactional Event publication
`warehouseaggregatemodel_idempotency`	duplicate-write protection
`warehouseaggregatemodel_exception`	exception, claim, dispute, or hold
`warehouseaggregatemodel_correction`	correction and supersession lineage
`warehouseaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Warehouse` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Warehouse — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-06-02`
Subject	`WarehouseContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Warehouse
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Warehouse**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Warehouse**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WarehouseContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WarehouseContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWarehouseContractModel`
- `ValidateWarehouseContractModel`
- `ApproveWarehouseContractModel`
- `ActivateWarehouseContractModel`
- `SuspendWarehouseContractModel`
- `CorrectWarehouseContractModel`
- `RetireWarehouseContractModel`

Reference Events:

- `WarehouseContractModelCreated`
- `WarehouseContractModelValidated`
- `WarehouseContractModelApproved`
- `WarehouseContractModelActivated`
- `WarehouseContractModelSuspended`
- `WarehouseContractModelCorrected`
- `WarehouseContractModelRetired`

Reference queries:

- `GetWarehouseContractModel`
- `ListWarehouseContractModelRecords`
- `GetWarehouseContractModelTimeline`
- `GetWarehouseContractModelEvidence`
- `GetWarehouseContractModelDecisionPackage`
- `GetWarehouseContractModelExceptions`
- `GetWarehouseContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/warehousecontractmodel/commands
GET  /api/v1/warehousecontractmodel/{id}
GET  /api/v1/warehousecontractmodel/{id}/timeline
GET  /api/v1/warehousecontractmodel/{id}/evidence
GET  /api/v1/warehousecontractmodel/{id}/decisions
POST /api/v1/warehousecontractmodel/{id}/exceptions
POST /api/v1/warehousecontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`warehousecontractmodel_aggregate`	authoritative subject
`warehousecontractmodel_version`	immutable submitted, approved, issued, or signed versions
`warehousecontractmodel_relationship`	governed relationships
`warehousecontractmodel_decision`	binding Decisions and rationale
`warehousecontractmodel_evidence_link`	provenance and evidence relationships
`warehousecontractmodel_event_outbox`	transactional Event publication
`warehousecontractmodel_idempotency`	duplicate-write protection
`warehousecontractmodel_exception`	exception, claim, dispute, or hold
`warehousecontractmodel_correction`	correction and supersession lineage
`warehousecontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Warehouse` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Warehouse — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-06-03`
Subject	`WarehouseDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Warehouse
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Warehouse**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Warehouse**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WarehouseDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WarehouseDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWarehouseDataModel`
- `ValidateWarehouseDataModel`
- `ApproveWarehouseDataModel`
- `ActivateWarehouseDataModel`
- `SuspendWarehouseDataModel`
- `CorrectWarehouseDataModel`
- `RetireWarehouseDataModel`

Reference Events:

- `WarehouseDataModelCreated`
- `WarehouseDataModelValidated`
- `WarehouseDataModelApproved`
- `WarehouseDataModelActivated`
- `WarehouseDataModelSuspended`
- `WarehouseDataModelCorrected`
- `WarehouseDataModelRetired`

Reference queries:

- `GetWarehouseDataModel`
- `ListWarehouseDataModelRecords`
- `GetWarehouseDataModelTimeline`
- `GetWarehouseDataModelEvidence`
- `GetWarehouseDataModelDecisionPackage`
- `GetWarehouseDataModelExceptions`
- `GetWarehouseDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/warehousedatamodel/commands
GET  /api/v1/warehousedatamodel/{id}
GET  /api/v1/warehousedatamodel/{id}/timeline
GET  /api/v1/warehousedatamodel/{id}/evidence
GET  /api/v1/warehousedatamodel/{id}/decisions
POST /api/v1/warehousedatamodel/{id}/exceptions
POST /api/v1/warehousedatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`warehousedatamodel_aggregate`	authoritative subject
`warehousedatamodel_version`	immutable submitted, approved, issued, or signed versions
`warehousedatamodel_relationship`	governed relationships
`warehousedatamodel_decision`	binding Decisions and rationale
`warehousedatamodel_evidence_link`	provenance and evidence relationships
`warehousedatamodel_event_outbox`	transactional Event publication
`warehousedatamodel_idempotency`	duplicate-write protection
`warehousedatamodel_exception`	exception, claim, dispute, or hold
`warehousedatamodel_correction`	correction and supersession lineage
`warehousedatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Warehouse` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Warehouse — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-06-04`
Subject	`WarehouseOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Warehouse
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Warehouse**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Warehouse**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`WarehouseOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`WarehouseOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
WarehouseOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateWarehouseOperatingProfile`
- `ValidateWarehouseOperatingProfile`
- `ApproveWarehouseOperatingProfile`
- `ActivateWarehouseOperatingProfile`
- `SuspendWarehouseOperatingProfile`
- `CorrectWarehouseOperatingProfile`
- `RetireWarehouseOperatingProfile`

Reference Events:

- `WarehouseOperatingProfileCreated`
- `WarehouseOperatingProfileValidated`
- `WarehouseOperatingProfileApproved`
- `WarehouseOperatingProfileActivated`
- `WarehouseOperatingProfileSuspended`
- `WarehouseOperatingProfileCorrected`
- `WarehouseOperatingProfileRetired`

Reference queries:

- `GetWarehouseOperatingProfile`
- `ListWarehouseOperatingProfileRecords`
- `GetWarehouseOperatingProfileTimeline`
- `GetWarehouseOperatingProfileEvidence`
- `GetWarehouseOperatingProfileDecisionPackage`
- `GetWarehouseOperatingProfileExceptions`
- `GetWarehouseOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/warehouseoperatingprofile/commands
GET  /api/v1/warehouseoperatingprofile/{id}
GET  /api/v1/warehouseoperatingprofile/{id}/timeline
GET  /api/v1/warehouseoperatingprofile/{id}/evidence
GET  /api/v1/warehouseoperatingprofile/{id}/decisions
POST /api/v1/warehouseoperatingprofile/{id}/exceptions
POST /api/v1/warehouseoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`warehouseoperatingprofile_aggregate`	authoritative subject
`warehouseoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`warehouseoperatingprofile_relationship`	governed relationships
`warehouseoperatingprofile_decision`	binding Decisions and rationale
`warehouseoperatingprofile_evidence_link`	provenance and evidence relationships
`warehouseoperatingprofile_event_outbox`	transactional Event publication
`warehouseoperatingprofile_idempotency`	duplicate-write protection
`warehouseoperatingprofile_exception`	exception, claim, dispute, or hold
`warehouseoperatingprofile_correction`	correction and supersession lineage
`warehouseoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Warehouse` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Shipping — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-07-00`
Subject	`ShippingDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Shipping
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Shipping**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Shipping**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ShippingDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ShippingDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateShippingDomain`
- `ValidateShippingDomain`
- `ApproveShippingDomain`
- `ActivateShippingDomain`
- `SuspendShippingDomain`
- `CorrectShippingDomain`
- `RetireShippingDomain`

Reference Events:

- `ShippingDomainCreated`
- `ShippingDomainValidated`
- `ShippingDomainApproved`
- `ShippingDomainActivated`
- `ShippingDomainSuspended`
- `ShippingDomainCorrected`
- `ShippingDomainRetired`

Reference queries:

- `GetShippingDomain`
- `ListShippingDomainRecords`
- `GetShippingDomainTimeline`
- `GetShippingDomainEvidence`
- `GetShippingDomainDecisionPackage`
- `GetShippingDomainExceptions`
- `GetShippingDomainAuditTrail`

Reference API surface:

```
POST /api/v1/shippingdomain/commands
GET  /api/v1/shippingdomain/{id}
GET  /api/v1/shippingdomain/{id}/timeline
GET  /api/v1/shippingdomain/{id}/evidence
GET  /api/v1/shippingdomain/{id}/decisions
POST /api/v1/shippingdomain/{id}/exceptions
POST /api/v1/shippingdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`shippingdomain_aggregate`	authoritative subject
`shippingdomain_version`	immutable submitted, approved, issued, or signed versions
`shippingdomain_relationship`	governed relationships
`shippingdomain_decision`	binding Decisions and rationale
`shippingdomain_evidence_link`	provenance and evidence relationships
`shippingdomain_event_outbox`	transactional Event publication
`shippingdomain_idempotency`	duplicate-write protection
`shippingdomain_exception`	exception, claim, dispute, or hold
`shippingdomain_correction`	correction and supersession lineage
`shippingdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Shipping` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Shipping — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-07-01`
Subject	`ShippingAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Shipping
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Shipping**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Shipping**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ShippingAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ShippingAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateShippingAggregateModel`
- `ValidateShippingAggregateModel`
- `ApproveShippingAggregateModel`
- `ActivateShippingAggregateModel`
- `SuspendShippingAggregateModel`
- `CorrectShippingAggregateModel`
- `RetireShippingAggregateModel`

Reference Events:

- `ShippingAggregateModelCreated`
- `ShippingAggregateModelValidated`
- `ShippingAggregateModelApproved`
- `ShippingAggregateModelActivated`
- `ShippingAggregateModelSuspended`
- `ShippingAggregateModelCorrected`
- `ShippingAggregateModelRetired`

Reference queries:

- `GetShippingAggregateModel`
- `ListShippingAggregateModelRecords`
- `GetShippingAggregateModelTimeline`
- `GetShippingAggregateModelEvidence`
- `GetShippingAggregateModelDecisionPackage`
- `GetShippingAggregateModelExceptions`
- `GetShippingAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/shippingaggregatemodel/commands
GET  /api/v1/shippingaggregatemodel/{id}
GET  /api/v1/shippingaggregatemodel/{id}/timeline
GET  /api/v1/shippingaggregatemodel/{id}/evidence
GET  /api/v1/shippingaggregatemodel/{id}/decisions
POST /api/v1/shippingaggregatemodel/{id}/exceptions
POST /api/v1/shippingaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`shippingaggregatemodel_aggregate`	authoritative subject
`shippingaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`shippingaggregatemodel_relationship`	governed relationships
`shippingaggregatemodel_decision`	binding Decisions and rationale
`shippingaggregatemodel_evidence_link`	provenance and evidence relationships
`shippingaggregatemodel_event_outbox`	transactional Event publication
`shippingaggregatemodel_idempotency`	duplicate-write protection
`shippingaggregatemodel_exception`	exception, claim, dispute, or hold
`shippingaggregatemodel_correction`	correction and supersession lineage
`shippingaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Shipping` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Shipping — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-07-02`
Subject	`ShippingContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Shipping
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Shipping**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Shipping**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ShippingContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ShippingContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateShippingContractModel`
- `ValidateShippingContractModel`
- `ApproveShippingContractModel`
- `ActivateShippingContractModel`
- `SuspendShippingContractModel`
- `CorrectShippingContractModel`
- `RetireShippingContractModel`

Reference Events:

- `ShippingContractModelCreated`
- `ShippingContractModelValidated`
- `ShippingContractModelApproved`
- `ShippingContractModelActivated`
- `ShippingContractModelSuspended`
- `ShippingContractModelCorrected`
- `ShippingContractModelRetired`

Reference queries:

- `GetShippingContractModel`
- `ListShippingContractModelRecords`
- `GetShippingContractModelTimeline`
- `GetShippingContractModelEvidence`
- `GetShippingContractModelDecisionPackage`
- `GetShippingContractModelExceptions`
- `GetShippingContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/shippingcontractmodel/commands
GET  /api/v1/shippingcontractmodel/{id}
GET  /api/v1/shippingcontractmodel/{id}/timeline
GET  /api/v1/shippingcontractmodel/{id}/evidence
GET  /api/v1/shippingcontractmodel/{id}/decisions
POST /api/v1/shippingcontractmodel/{id}/exceptions
POST /api/v1/shippingcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`shippingcontractmodel_aggregate`	authoritative subject
`shippingcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`shippingcontractmodel_relationship`	governed relationships
`shippingcontractmodel_decision`	binding Decisions and rationale
`shippingcontractmodel_evidence_link`	provenance and evidence relationships
`shippingcontractmodel_event_outbox`	transactional Event publication
`shippingcontractmodel_idempotency`	duplicate-write protection
`shippingcontractmodel_exception`	exception, claim, dispute, or hold
`shippingcontractmodel_correction`	correction and supersession lineage
`shippingcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Shipping` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Shipping — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-07-03`
Subject	`ShippingDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Shipping
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Shipping**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Shipping**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ShippingDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ShippingDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateShippingDataModel`
- `ValidateShippingDataModel`
- `ApproveShippingDataModel`
- `ActivateShippingDataModel`
- `SuspendShippingDataModel`
- `CorrectShippingDataModel`
- `RetireShippingDataModel`

Reference Events:

- `ShippingDataModelCreated`
- `ShippingDataModelValidated`
- `ShippingDataModelApproved`
- `ShippingDataModelActivated`
- `ShippingDataModelSuspended`
- `ShippingDataModelCorrected`
- `ShippingDataModelRetired`

Reference queries:

- `GetShippingDataModel`
- `ListShippingDataModelRecords`
- `GetShippingDataModelTimeline`
- `GetShippingDataModelEvidence`
- `GetShippingDataModelDecisionPackage`
- `GetShippingDataModelExceptions`
- `GetShippingDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/shippingdatamodel/commands
GET  /api/v1/shippingdatamodel/{id}
GET  /api/v1/shippingdatamodel/{id}/timeline
GET  /api/v1/shippingdatamodel/{id}/evidence
GET  /api/v1/shippingdatamodel/{id}/decisions
POST /api/v1/shippingdatamodel/{id}/exceptions
POST /api/v1/shippingdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`shippingdatamodel_aggregate`	authoritative subject
`shippingdatamodel_version`	immutable submitted, approved, issued, or signed versions
`shippingdatamodel_relationship`	governed relationships
`shippingdatamodel_decision`	binding Decisions and rationale
`shippingdatamodel_evidence_link`	provenance and evidence relationships
`shippingdatamodel_event_outbox`	transactional Event publication
`shippingdatamodel_idempotency`	duplicate-write protection
`shippingdatamodel_exception`	exception, claim, dispute, or hold
`shippingdatamodel_correction`	correction and supersession lineage
`shippingdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Shipping` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Shipping — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-07-04`
Subject	`ShippingOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Shipping
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Shipping**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Shipping**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ShippingOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ShippingOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ShippingOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateShippingOperatingProfile`
- `ValidateShippingOperatingProfile`
- `ApproveShippingOperatingProfile`
- `ActivateShippingOperatingProfile`
- `SuspendShippingOperatingProfile`
- `CorrectShippingOperatingProfile`
- `RetireShippingOperatingProfile`

Reference Events:

- `ShippingOperatingProfileCreated`
- `ShippingOperatingProfileValidated`
- `ShippingOperatingProfileApproved`
- `ShippingOperatingProfileActivated`
- `ShippingOperatingProfileSuspended`
- `ShippingOperatingProfileCorrected`
- `ShippingOperatingProfileRetired`

Reference queries:

- `GetShippingOperatingProfile`
- `ListShippingOperatingProfileRecords`
- `GetShippingOperatingProfileTimeline`
- `GetShippingOperatingProfileEvidence`
- `GetShippingOperatingProfileDecisionPackage`
- `GetShippingOperatingProfileExceptions`
- `GetShippingOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/shippingoperatingprofile/commands
GET  /api/v1/shippingoperatingprofile/{id}
GET  /api/v1/shippingoperatingprofile/{id}/timeline
GET  /api/v1/shippingoperatingprofile/{id}/evidence
GET  /api/v1/shippingoperatingprofile/{id}/decisions
POST /api/v1/shippingoperatingprofile/{id}/exceptions
POST /api/v1/shippingoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`shippingoperatingprofile_aggregate`	authoritative subject
`shippingoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`shippingoperatingprofile_relationship`	governed relationships
`shippingoperatingprofile_decision`	binding Decisions and rationale
`shippingoperatingprofile_evidence_link`	provenance and evidence relationships
`shippingoperatingprofile_event_outbox`	transactional Event publication
`shippingoperatingprofile_idempotency`	duplicate-write protection
`shippingoperatingprofile_exception`	exception, claim, dispute, or hold
`shippingoperatingprofile_correction`	correction and supersession lineage
`shippingoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Shipping` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter’s evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Regulatory — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-08-00`
Subject	`RegulatoryDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Regulatory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Regulatory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Regulatory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RegulatoryDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RegulatoryDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRegulatoryDomain`
- `ValidateRegulatoryDomain`
- `ApproveRegulatoryDomain`
- `ActivateRegulatoryDomain`
- `SuspendRegulatoryDomain`
- `CorrectRegulatoryDomain`
- `RetireRegulatoryDomain`

Reference Events:

- `RegulatoryDomainCreated`
- `RegulatoryDomainValidated`
- `RegulatoryDomainApproved`
- `RegulatoryDomainActivated`
- `RegulatoryDomainSuspended`
- `RegulatoryDomainCorrected`
- `RegulatoryDomainRetired`

Reference queries:

- `GetRegulatoryDomain`
- `ListRegulatoryDomainRecords`
- `GetRegulatoryDomainTimeline`
- `GetRegulatoryDomainEvidence`
- `GetRegulatoryDomainDecisionPackage`
- `GetRegulatoryDomainExceptions`
- `GetRegulatoryDomainAuditTrail`

Reference API surface:

```
POST /api/v1/regulatorydomain/commands
GET  /api/v1/regulatorydomain/{id}
GET  /api/v1/regulatorydomain/{id}/timeline
GET  /api/v1/regulatorydomain/{id}/evidence
GET  /api/v1/regulatorydomain/{id}/decisions
POST /api/v1/regulatorydomain/{id}/exceptions
POST /api/v1/regulatorydomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`regulatorydomain_aggregate`	authoritative subject
`regulatorydomain_version`	immutable submitted, approved, issued, or signed versions
`regulatorydomain_relationship`	governed relationships
`regulatorydomain_decision`	binding Decisions and rationale
`regulatorydomain_evidence_link`	provenance and evidence relationships
`regulatorydomain_event_outbox`	transactional Event publication
`regulatorydomain_idempotency`	duplicate-write protection
`regulatorydomain_exception`	exception, claim, dispute, or hold
`regulatorydomain_correction`	correction and supersession lineage
`regulatorydomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Regulatory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Regulatory — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-08-01`
Subject	`RegulatoryAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Regulatory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, life-cycle state, effective time, correction, and concurrency for Regulatory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Regulatory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RegulatoryAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RegulatoryAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRegulatoryAggregateModel`
- `ValidateRegulatoryAggregateModel`
- `ApproveRegulatoryAggregateModel`
- `ActivateRegulatoryAggregateModel`
- `SuspendRegulatoryAggregateModel`
- `CorrectRegulatoryAggregateModel`
- `RetireRegulatoryAggregateModel`

Reference Events:

- `RegulatoryAggregateModelCreated`
- `RegulatoryAggregateModelValidated`
- `RegulatoryAggregateModelApproved`
- `RegulatoryAggregateModelActivated`
- `RegulatoryAggregateModelSuspended`
- `RegulatoryAggregateModelCorrected`
- `RegulatoryAggregateModelRetired`

Reference queries:

- `GetRegulatoryAggregateModel`
- `ListRegulatoryAggregateModelRecords`
- `GetRegulatoryAggregateModelTimeline`
- `GetRegulatoryAggregateModelEvidence`
- `GetRegulatoryAggregateModelDecisionPackage`
- `GetRegulatoryAggregateModelExceptions`
- `GetRegulatoryAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/regulatoryaggregatemodel/commands
GET  /api/v1/regulatoryaggregatemodel/{id}
GET  /api/v1/regulatoryaggregatemodel/{id}/timeline
GET  /api/v1/regulatoryaggregatemodel/{id}/evidence
GET  /api/v1/regulatoryaggregatemodel/{id}/decisions
POST /api/v1/regulatoryaggregatemodel/{id}/exceptions
POST /api/v1/regulatoryaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`regulatoryaggregatemodel_aggregate`	authoritative subject
`regulatoryaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`regulatoryaggregatemodel_relationship`	governed relationships
`regulatoryaggregatemodel_decision`	binding Decisions and rationale
`regulatoryaggregatemodel_evidence_link`	provenance and evidence relationships
`regulatoryaggregatemodel_event_outbox`	transactional Event publication
`regulatoryaggregatemodel_idempotency`	duplicate-write protection
`regulatoryaggregatemodel_exception`	exception, claim, dispute, or hold
`regulatoryaggregatemodel_correction`	correction and supersession lineage
`regulatoryaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Regulatory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Regulatory — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-08-02`
Subject	`RegulatoryContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Regulatory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Regulatory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Regulatory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RegulatoryContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RegulatoryContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRegulatoryContractModel`
- `ValidateRegulatoryContractModel`
- `ApproveRegulatoryContractModel`
- `ActivateRegulatoryContractModel`
- `SuspendRegulatoryContractModel`
- `CorrectRegulatoryContractModel`
- `RetireRegulatoryContractModel`

Reference Events:

- `RegulatoryContractModelCreated`
- `RegulatoryContractModelValidated`
- `RegulatoryContractModelApproved`
- `RegulatoryContractModelActivated`
- `RegulatoryContractModelSuspended`
- `RegulatoryContractModelCorrected`
- `RegulatoryContractModelRetired`

Reference queries:

- `GetRegulatoryContractModel`
- `ListRegulatoryContractModelRecords`
- `GetRegulatoryContractModelTimeline`
- `GetRegulatoryContractModelEvidence`
- `GetRegulatoryContractModelDecisionPackage`
- `GetRegulatoryContractModelExceptions`
- `GetRegulatoryContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/regulatorycontractmodel/commands
GET  /api/v1/regulatorycontractmodel/{id}
GET  /api/v1/regulatorycontractmodel/{id}/timeline
GET  /api/v1/regulatorycontractmodel/{id}/evidence
GET  /api/v1/regulatorycontractmodel/{id}/decisions
POST /api/v1/regulatorycontractmodel/{id}/exceptions
POST /api/v1/regulatorycontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`regulatorycontractmodel_aggregate`	authoritative subject
`regulatorycontractmodel_version`	immutable submitted, approved, issued, or signed versions
`regulatorycontractmodel_relationship`	governed relationships
`regulatorycontractmodel_decision`	binding Decisions and rationale
`regulatorycontractmodel_evidence_link`	provenance and evidence relationships
`regulatorycontractmodel_event_outbox`	transactional Event publication
`regulatorycontractmodel_idempotency`	duplicate-write protection
`regulatorycontractmodel_exception`	exception, claim, dispute, or hold
`regulatorycontractmodel_correction`	correction and supersession lineage
`regulatorycontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Regulatory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Regulatory — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-08-03`
Subject	`RegulatoryDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Regulatory
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Regulatory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Regulatory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RegulatoryDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RegulatoryDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRegulatoryDataModel`
- `ValidateRegulatoryDataModel`
- `ApproveRegulatoryDataModel`
- `ActivateRegulatoryDataModel`
- `SuspendRegulatoryDataModel`
- `CorrectRegulatoryDataModel`
- `RetireRegulatoryDataModel`

Reference Events:

- `RegulatoryDataModelCreated`
- `RegulatoryDataModelValidated`
- `RegulatoryDataModelApproved`
- `RegulatoryDataModelActivated`
- `RegulatoryDataModelSuspended`
- `RegulatoryDataModelCorrected`
- `RegulatoryDataModelRetired`

Reference queries:

- `GetRegulatoryDataModel`
- `ListRegulatoryDataModelRecords`
- `GetRegulatoryDataModelTimeline`
- `GetRegulatoryDataModelEvidence`
- `GetRegulatoryDataModelDecisionPackage`
- `GetRegulatoryDataModelExceptions`
- `GetRegulatoryDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/regulatorydatamodel/commands
GET  /api/v1/regulatorydatamodel/{id}
GET  /api/v1/regulatorydatamodel/{id}/timeline
GET  /api/v1/regulatorydatamodel/{id}/evidence
GET  /api/v1/regulatorydatamodel/{id}/decisions
POST /api/v1/regulatorydatamodel/{id}/exceptions
POST /api/v1/regulatorydatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`regulatorydatamodel_aggregate`	authoritative subject
`regulatorydatamodel_version`	immutable submitted, approved, issued, or signed versions
`regulatorydatamodel_relationship`	governed relationships
`regulatorydatamodel_decision`	binding Decisions and rationale
`regulatorydatamodel_evidence_link`	provenance and evidence relationships
`regulatorydatamodel_event_outbox`	transactional Event publication
`regulatorydatamodel_idempotency`	duplicate-write protection
`regulatorydatamodel_exception`	exception, claim, dispute, or hold
`regulatorydatamodel_correction`	correction and supersession lineage
`regulatorydatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Regulatory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Regulatory — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	V07-08-04
Subject	RegulatoryOperatingProfile
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / Regulatory
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Regulatory**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Regulatory**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`RegulatoryOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`RegulatoryOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
RegulatoryOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateRegulatoryOperatingProfile`
- `ValidateRegulatoryOperatingProfile`
- `ApproveRegulatoryOperatingProfile`
- `ActivateRegulatoryOperatingProfile`
- `SuspendRegulatoryOperatingProfile`
- `CorrectRegulatoryOperatingProfile`
- `RetireRegulatoryOperatingProfile`

Reference Events:

- `RegulatoryOperatingProfileCreated`
- `RegulatoryOperatingProfileValidated`
- `RegulatoryOperatingProfileApproved`
- `RegulatoryOperatingProfileActivated`
- `RegulatoryOperatingProfileSuspended`
- `RegulatoryOperatingProfileCorrected`
- `RegulatoryOperatingProfileRetired`

Reference queries:

- `GetRegulatoryOperatingProfile`
- `ListRegulatoryOperatingProfileRecords`
- `GetRegulatoryOperatingProfileTimeline`
- `GetRegulatoryOperatingProfileEvidence`
- `GetRegulatoryOperatingProfileDecisionPackage`
- `GetRegulatoryOperatingProfileExceptions`
- `GetRegulatoryOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/regulatoryoperatingprofile/commands
GET  /api/v1/regulatoryoperatingprofile/{id}
GET  /api/v1/regulatoryoperatingprofile/{id}/timeline
GET  /api/v1/regulatoryoperatingprofile/{id}/evidence
GET  /api/v1/regulatoryoperatingprofile/{id}/decisions
POST /api/v1/regulatoryoperatingprofile/{id}/exceptions
POST /api/v1/regulatoryoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`regulatoryoperatingprofile_aggregate`	authoritative subject
`regulatoryoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`regulatoryoperatingprofile_relationship`	governed relationships
`regulatoryoperatingprofile_decision`	binding Decisions and rationale
`regulatoryoperatingprofile_evidence_link`	provenance and evidence relationships
`regulatoryoperatingprofile_event_outbox`	transactional Event publication
`regulatoryoperatingprofile_idempotency`	duplicate-write protection
`regulatoryoperatingprofile_exception`	exception, claim, dispute, or hold
`regulatoryoperatingprofile_correction`	correction and supersession lineage
`regulatoryoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Regulatory` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Production Fulfilment — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-09-00`
Subject	`ProductionFulfilmentDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Production Fulfilment
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Production Fulfilment**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Production Fulfilment**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProductionFulfilmentDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProductionFulfilmentDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProductionFulfilmentDomain`
- `ValidateProductionFulfilmentDomain`
- `ApproveProductionFulfilmentDomain`
- `ActivateProductionFulfilmentDomain`
- `SuspendProductionFulfilmentDomain`
- `CorrectProductionFulfilmentDomain`
- `RetireProductionFulfilmentDomain`

Reference Events:

- `ProductionFulfilmentDomainCreated`
- `ProductionFulfilmentDomainValidated`
- `ProductionFulfilmentDomainApproved`
- `ProductionFulfilmentDomainActivated`
- `ProductionFulfilmentDomainSuspended`
- `ProductionFulfilmentDomainCorrected`
- `ProductionFulfilmentDomainRetired`

Reference queries:

- `GetProductionFulfilmentDomain`
- `ListProductionFulfilmentDomainRecords`
- `GetProductionFulfilmentDomainTimeline`
- `GetProductionFulfilmentDomainEvidence`
- `GetProductionFulfilmentDomainDecisionPackage`
- `GetProductionFulfilmentDomainExceptions`
- `GetProductionFulfilmentDomainAuditTrail`

Reference API surface:

```
POST /api/v1/productionfulfilmentdomain/commands
GET  /api/v1/productionfulfilmentdomain/{id}
GET  /api/v1/productionfulfilmentdomain/{id}/timeline
GET  /api/v1/productionfulfilmentdomain/{id}/evidence
GET  /api/v1/productionfulfilmentdomain/{id}/decisions
POST /api/v1/productionfulfilmentdomain/{id}/exceptions
POST /api/v1/productionfulfilmentdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
C -- Yes --> E[Record Decision or execution Intent]
E --> F[Execute Domain command or external request]
F --> G{Outcome known?}
G -- Success --> H[Record fact and publish Event]
G -- Failure --> I[Record failure and compensation options]
G -- Unknown --> J[Enter Outcome Unknown]
J --> K[Query source or wait for verified callback]
K --> G
H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`productionfulfilmentdomain_aggregate`	authoritative subject
`productionfulfilmentdomain_version`	immutable submitted, approved, issued, or signed versions
`productionfulfilmentdomain_relationship`	governed relationships
`productionfulfilmentdomain_decision`	binding Decisions and rationale
`productionfulfilmentdomain_evidence_link`	provenance and evidence relationships
`productionfulfilmentdomain_event_outbox`	transactional Event publication
`productionfulfilmentdomain_idempotency`	duplicate-write protection
`productionfulfilmentdomain_exception`	exception, claim, dispute, or hold
`productionfulfilmentdomain_correction`	correction and supersession lineage
`productionfulfilmentdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Production Fulfilment` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Production Fulfilment — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-09-01`
Subject	`ProductionFulfilmentAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Production Fulfilment
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Production Fulfilment**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Production Fulfilment**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProductionFulfilmentAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProductionFulfilmentAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProductionFulfilmentAggregateModel`
- `ValidateProductionFulfilmentAggregateModel`
- `ApproveProductionFulfilmentAggregateModel`
- `ActivateProductionFulfilmentAggregateModel`
- `SuspendProductionFulfilmentAggregateModel`
- `CorrectProductionFulfilmentAggregateModel`
- `RetireProductionFulfilmentAggregateModel`

Reference Events:

- `ProductionFulfilmentAggregateModelCreated`
- `ProductionFulfilmentAggregateModelValidated`
- `ProductionFulfilmentAggregateModelApproved`
- `ProductionFulfilmentAggregateModelActivated`
- `ProductionFulfilmentAggregateModelSuspended`
- `ProductionFulfilmentAggregateModelCorrected`
- `ProductionFulfilmentAggregateModelRetired`

Reference queries:

- `GetProductionFulfilmentAggregateModel`
- `ListProductionFulfilmentAggregateModelRecords`
- `GetProductionFulfilmentAggregateModelTimeline`
- `GetProductionFulfilmentAggregateModelEvidence`
- `GetProductionFulfilmentAggregateModelDecisionPackage`
- `GetProductionFulfilmentAggregateModelExceptions`
- `GetProductionFulfilmentAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/productionfulfilmentaggregatemodel/commands
GET  /api/v1/productionfulfilmentaggregatemodel/{id}
GET  /api/v1/productionfulfilmentaggregatemodel/{id}/timeline
GET  /api/v1/productionfulfilmentaggregatemodel/{id}/evidence
GET  /api/v1/productionfulfilmentaggregatemodel/{id}/decisions
POST /api/v1/productionfulfilmentaggregatemodel/{id}/exceptions
POST /api/v1/productionfulfilmentaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`productionfulfilmentaggregatemodel_aggregate`	authoritative subject
`productionfulfilmentaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`productionfulfilmentaggregatemodel_relationship`	governed relationships
`productionfulfilmentaggregatemodel_decision`	binding Decisions and rationale
`productionfulfilmentaggregatemodel_evidence_link`	provenance and evidence relationships
`productionfulfilmentaggregatemodel_event_outbox`	transactional Event publication
`productionfulfilmentaggregatemodel_idempotency`	duplicate-write protection
`productionfulfilmentaggregatemodel_exception`	exception, claim, dispute, or hold
`productionfulfilmentaggregatemodel_correction`	correction and supersession lineage
`productionfulfilmentaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Production Fulfilment` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Production Fulfilment — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-09-02`
Subject	`ProductionFulfilmentContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Production Fulfilment
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Production Fulfilment**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Production Fulfilment**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProductionFulfilmentContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProductionFulfilmentContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProductionFulfilmentContractModel`
- `ValidateProductionFulfilmentContractModel`
- `ApproveProductionFulfilmentContractModel`
- `ActivateProductionFulfilmentContractModel`
- `SuspendProductionFulfilmentContractModel`
- `CorrectProductionFulfilmentContractModel`
- `RetireProductionFulfilmentContractModel`

Reference Events:

- `ProductionFulfilmentContractModelCreated`
- `ProductionFulfilmentContractModelValidated`
- `ProductionFulfilmentContractModelApproved`
- `ProductionFulfilmentContractModelActivated`
- `ProductionFulfilmentContractModelSuspended`
- `ProductionFulfilmentContractModelCorrected`
- `ProductionFulfilmentContractModelRetired`

Reference queries:

- `GetProductionFulfilmentContractModel`
- `ListProductionFulfilmentContractModelRecords`
- `GetProductionFulfilmentContractModelTimeline`
- `GetProductionFulfilmentContractModelEvidence`
- `GetProductionFulfilmentContractModelDecisionPackage`
- `GetProductionFulfilmentContractModelExceptions`
- `GetProductionFulfilmentContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/productionfulfilmentcontractmodel/commands
GET  /api/v1/productionfulfilmentcontractmodel/{id}
GET  /api/v1/productionfulfilmentcontractmodel/{id}/timeline
GET  /api/v1/productionfulfilmentcontractmodel/{id}/evidence
GET  /api/v1/productionfulfilmentcontractmodel/{id}/decisions
POST /api/v1/productionfulfilmentcontractmodel/{id}/exceptions
POST /api/v1/productionfulfilmentcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`productionfulfilmentcontractmodel_aggregate`	authoritative subject
`productionfulfilmentcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`productionfulfilmentcontractmodel_relationship`	governed relationships
`productionfulfilmentcontractmodel_decision`	binding Decisions and rationale
`productionfulfilmentcontractmodel_evidence_link`	provenance and evidence relationships
`productionfulfilmentcontractmodel_event_outbox`	transactional Event publication
`productionfulfilmentcontractmodel_idempotency`	duplicate-write protection
`productionfulfilmentcontractmodel_exception`	exception, claim, dispute, or hold
`productionfulfilmentcontractmodel_correction`	correction and supersession lineage
`productionfulfilmentcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Production Fulfilment` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Production Fulfilment — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-09-03`
Subject	`ProductionFulfilmentDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Production Fulfilment
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Production Fulfilment**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Production Fulfilment**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProductionFulfilmentDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProductionFulfilmentDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProductionFulfilmentDataModel`
- `ValidateProductionFulfilmentDataModel`
- `ApproveProductionFulfilmentDataModel`
- `ActivateProductionFulfilmentDataModel`
- `SuspendProductionFulfilmentDataModel`
- `CorrectProductionFulfilmentDataModel`
- `RetireProductionFulfilmentDataModel`

Reference Events:

- `ProductionFulfilmentDataModelCreated`
- `ProductionFulfilmentDataModelValidated`
- `ProductionFulfilmentDataModelApproved`
- `ProductionFulfilmentDataModelActivated`
- `ProductionFulfilmentDataModelSuspended`
- `ProductionFulfilmentDataModelCorrected`
- `ProductionFulfilmentDataModelRetired`

Reference queries:

- `GetProductionFulfilmentDataModel`
- `ListProductionFulfilmentDataModelRecords`
- `GetProductionFulfilmentDataModelTimeline`
- `GetProductionFulfilmentDataModelEvidence`
- `GetProductionFulfilmentDataModelDecisionPackage`
- `GetProductionFulfilmentDataModelExceptions`
- `GetProductionFulfilmentDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/productionfulfilmentdatamodel/commands
GET  /api/v1/productionfulfilmentdatamodel/{id}
GET  /api/v1/productionfulfilmentdatamodel/{id}/timeline
GET  /api/v1/productionfulfilmentdatamodel/{id}/evidence
GET  /api/v1/productionfulfilmentdatamodel/{id}/decisions
POST /api/v1/productionfulfilmentdatamodel/{id}/exceptions
POST /api/v1/productionfulfilmentdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`productionfulfilmentdatamodel_aggregate`	authoritative subject
`productionfulfilmentdatamodel_version`	immutable submitted, approved, issued, or signed versions
`productionfulfilmentdatamodel_relationship`	governed relationships
`productionfulfilmentdatamodel_decision`	binding Decisions and rationale
`productionfulfilmentdatamodel_evidence_link`	provenance and evidence relationships
`productionfulfilmentdatamodel_event_outbox`	transactional Event publication
`productionfulfilmentdatamodel_idempotency`	duplicate-write protection
`productionfulfilmentdatamodel_exception`	exception, claim, dispute, or hold
`productionfulfilmentdatamodel_correction`	correction and supersession lineage
`productionfulfilmentdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Production Fulfilment` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Production Fulfilment — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-09-04`
Subject	`ProductionFulfilmentOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Production Fulfilment
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Production Fulfilment**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Production Fulfilment**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ProductionFulfilmentOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ProductionFulfilmentOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ProductionFulfilmentOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateProductionFulfilmentOperatingProfile`
- `ValidateProductionFulfilmentOperatingProfile`
- `ApproveProductionFulfilmentOperatingProfile`
- `ActivateProductionFulfilmentOperatingProfile`
- `SuspendProductionFulfilmentOperatingProfile`
- `CorrectProductionFulfilmentOperatingProfile`
- `RetireProductionFulfilmentOperatingProfile`

Reference Events:

- `ProductionFulfilmentOperatingProfileCreated`
- `ProductionFulfilmentOperatingProfileValidated`
- `ProductionFulfilmentOperatingProfileApproved`
- `ProductionFulfilmentOperatingProfileActivated`
- `ProductionFulfilmentOperatingProfileSuspended`
- `ProductionFulfilmentOperatingProfileCorrected`
- `ProductionFulfilmentOperatingProfileRetired`

Reference queries:

- `GetProductionFulfilmentOperatingProfile`
- `ListProductionFulfilmentOperatingProfileRecords`
- `GetProductionFulfilmentOperatingProfileTimeline`
- `GetProductionFulfilmentOperatingProfileEvidence`
- `GetProductionFulfilmentOperatingProfileDecisionPackage`
- `GetProductionFulfilmentOperatingProfileExceptions`
- `GetProductionFulfilmentOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/productionfulfilmentoperatingprofile/commands
GET  /api/v1/productionfulfilmentoperatingprofile/{id}
GET  /api/v1/productionfulfilmentoperatingprofile/{id}/timeline
GET  /api/v1/productionfulfilmentoperatingprofile/{id}/evidence
GET  /api/v1/productionfulfilmentoperatingprofile/{id}/decisions
POST /api/v1/productionfulfilmentoperatingprofile/{id}/exceptions
POST /api/v1/productionfulfilmentoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`productionfulfilmentoperatingprofile_aggregate`	authoritative subject
`productionfulfilmentoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`productionfulfilmentoperatingprofile_relationship`	governed relationships
`productionfulfilmentoperatingprofile_decision`	binding Decisions and rationale
`productionfulfilmentoperatingprofile_evidence_link`	provenance and evidence relationships
`productionfulfilmentoperatingprofile_event_outbox`	transactional Event publication
`productionfulfilmentoperatingprofile_idempotency`	duplicate-write protection
`productionfulfilmentoperatingprofile_exception`	exception, claim, dispute, or hold
`productionfulfilmentoperatingprofile_correction`	correction and supersession lineage
`productionfulfilmentoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Production Fulfilment` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Allocation — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-10-00`
Subject	`AllocationDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Allocation
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Allocation**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Allocation**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AllocationDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AllocationDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAllocationDomain`
- `ValidateAllocationDomain`
- `ApproveAllocationDomain`
- `ActivateAllocationDomain`
- `SuspendAllocationDomain`
- `CorrectAllocationDomain`
- `RetireAllocationDomain`

Reference Events:

- `AllocationDomainCreated`
- `AllocationDomainValidated`
- `AllocationDomainApproved`
- `AllocationDomainActivated`
- `AllocationDomainSuspended`
- `AllocationDomainCorrected`
- `AllocationDomainRetired`

Reference queries:

- `GetAllocationDomain`
- `ListAllocationDomainRecords`
- `GetAllocationDomainTimeline`
- `GetAllocationDomainEvidence`
- `GetAllocationDomainDecisionPackage`
- `GetAllocationDomainExceptions`
- `GetAllocationDomainAuditTrail`

Reference API surface:

```
POST /api/v1/allocationdomain/commands
GET  /api/v1/allocationdomain/{id}
GET  /api/v1/allocationdomain/{id}/timeline
GET  /api/v1/allocationdomain/{id}/evidence
GET  /api/v1/allocationdomain/{id}/decisions
POST /api/v1/allocationdomain/{id}/exceptions
POST /api/v1/allocationdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`allocationdomain_aggregate`	authoritative subject
`allocationdomain_version`	immutable submitted, approved, issued, or signed versions
`allocationdomain_relationship`	governed relationships
`allocationdomain_decision`	binding Decisions and rationale
`allocationdomain_evidence_link`	provenance and evidence relationships
`allocationdomain_event_outbox`	transactional Event publication
`allocationdomain_idempotency`	duplicate-write protection
`allocationdomain_exception`	exception, claim, dispute, or hold
`allocationdomain_correction`	correction and supersession lineage
`allocationdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Allocation` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Allocation — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-10-01`
Subject	`AllocationAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Allocation
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Allocation**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Allocation**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AllocationAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AllocationAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAllocationAggregateModel`
- `ValidateAllocationAggregateModel`
- `ApproveAllocationAggregateModel`
- `ActivateAllocationAggregateModel`
- `SuspendAllocationAggregateModel`
- `CorrectAllocationAggregateModel`
- `RetireAllocationAggregateModel`

Reference Events:

- `AllocationAggregateModelCreated`
- `AllocationAggregateModelValidated`
- `AllocationAggregateModelApproved`
- `AllocationAggregateModelActivated`
- `AllocationAggregateModelSuspended`
- `AllocationAggregateModelCorrected`
- `AllocationAggregateModelRetired`

Reference queries:

- `GetAllocationAggregateModel`
- `ListAllocationAggregateModelRecords`
- `GetAllocationAggregateModelTimeline`
- `GetAllocationAggregateModelEvidence`
- `GetAllocationAggregateModelDecisionPackage`
- `GetAllocationAggregateModelExceptions`
- `GetAllocationAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/allocationaggregatemodel/commands
GET  /api/v1/allocationaggregatemodel/{id}
GET  /api/v1/allocationaggregatemodel/{id}/timeline
GET  /api/v1/allocationaggregatemodel/{id}/evidence
GET  /api/v1/allocationaggregatemodel/{id}/decisions
POST /api/v1/allocationaggregatemodel/{id}/exceptions
POST /api/v1/allocationaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`allocationaggregatemodel_aggregate`	authoritative subject
`allocationaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`allocationaggregatemodel_relationship`	governed relationships
`allocationaggregatemodel_decision`	binding Decisions and rationale
`allocationaggregatemodel_evidence_link`	provenance and evidence relationships
`allocationaggregatemodel_event_outbox`	transactional Event publication
`allocationaggregatemodel_idempotency`	duplicate-write protection
`allocationaggregatemodel_exception`	exception, claim, dispute, or hold
`allocationaggregatemodel_correction`	correction and supersession lineage
`allocationaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Allocation` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Allocation — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	V07-10-02
Subject	AllocationContractModel
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / Allocation
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Allocation**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Allocation**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AllocationContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AllocationContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAllocationContractModel`
- `ValidateAllocationContractModel`
- `ApproveAllocationContractModel`
- `ActivateAllocationContractModel`
- `SuspendAllocationContractModel`
- `CorrectAllocationContractModel`
- `RetireAllocationContractModel`

Reference Events:

- `AllocationContractModelCreated`
- `AllocationContractModelValidated`
- `AllocationContractModelApproved`
- `AllocationContractModelActivated`
- `AllocationContractModelSuspended`
- `AllocationContractModelCorrected`
- `AllocationContractModelRetired`

Reference queries:

- `GetAllocationContractModel`
- `ListAllocationContractModelRecords`
- `GetAllocationContractModelTimeline`
- `GetAllocationContractModelEvidence`
- `GetAllocationContractModelDecisionPackage`
- `GetAllocationContractModelExceptions`
- `GetAllocationContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/allocationcontractmodel/commands
GET  /api/v1/allocationcontractmodel/{id}
GET  /api/v1/allocationcontractmodel/{id}/timeline
GET  /api/v1/allocationcontractmodel/{id}/evidence
GET  /api/v1/allocationcontractmodel/{id}/decisions
POST /api/v1/allocationcontractmodel/{id}/exceptions
POST /api/v1/allocationcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`allocationcontractmodel_aggregate`	authoritative subject
`allocationcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`allocationcontractmodel_relationship`	governed relationships
`allocationcontractmodel_decision`	binding Decisions and rationale
`allocationcontractmodel_evidence_link`	provenance and evidence relationships
`allocationcontractmodel_event_outbox`	transactional Event publication
`allocationcontractmodel_idempotency`	duplicate-write protection
`allocationcontractmodel_exception`	exception, claim, dispute, or hold
`allocationcontractmodel_correction`	correction and supersession lineage
`allocationcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Allocation` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Allocation — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-10-03`
Subject	`AllocationDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Allocation
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Allocation**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Allocation**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AllocationDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AllocationDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAllocationDataModel`
- `ValidateAllocationDataModel`
- `ApproveAllocationDataModel`
- `ActivateAllocationDataModel`
- `SuspendAllocationDataModel`
- `CorrectAllocationDataModel`
- `RetireAllocationDataModel`

Reference Events:

- `AllocationDataModelCreated`
- `AllocationDataModelValidated`
- `AllocationDataModelApproved`
- `AllocationDataModelActivated`
- `AllocationDataModelSuspended`
- `AllocationDataModelCorrected`
- `AllocationDataModelRetired`

Reference queries:

- `GetAllocationDataModel`
- `ListAllocationDataModelRecords`
- `GetAllocationDataModelTimeline`
- `GetAllocationDataModelEvidence`
- `GetAllocationDataModelDecisionPackage`
- `GetAllocationDataModelExceptions`
- `GetAllocationDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/allocationdatamodel/commands
GET  /api/v1/allocationdatamodel/{id}
GET  /api/v1/allocationdatamodel/{id}/timeline
GET  /api/v1/allocationdatamodel/{id}/evidence
GET  /api/v1/allocationdatamodel/{id}/decisions
POST /api/v1/allocationdatamodel/{id}/exceptions
POST /api/v1/allocationdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`allocationdatamodel_aggregate`	authoritative subject
`allocationdatamodel_version`	immutable submitted, approved, issued, or signed versions
`allocationdatamodel_relationship`	governed relationships
`allocationdatamodel_decision`	binding Decisions and rationale
`allocationdatamodel_evidence_link`	provenance and evidence relationships
`allocationdatamodel_event_outbox`	transactional Event publication
`allocationdatamodel_idempotency`	duplicate-write protection
`allocationdatamodel_exception`	exception, claim, dispute, or hold
`allocationdatamodel_correction`	correction and supersession lineage
`allocationdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Allocation` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Allocation — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-10-04`
Subject	`AllocationOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Allocation
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Allocation**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Allocation**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`AllocationOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`AllocationOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
AllocationOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateAllocationOperatingProfile`
- `ValidateAllocationOperatingProfile`
- `ApproveAllocationOperatingProfile`
- `ActivateAllocationOperatingProfile`
- `SuspendAllocationOperatingProfile`
- `CorrectAllocationOperatingProfile`
- `RetireAllocationOperatingProfile`

Reference Events:

- `AllocationOperatingProfileCreated`
- `AllocationOperatingProfileValidated`
- `AllocationOperatingProfileApproved`
- `AllocationOperatingProfileActivated`
- `AllocationOperatingProfileSuspended`
- `AllocationOperatingProfileCorrected`
- `AllocationOperatingProfileRetired`

Reference queries:

- `GetAllocationOperatingProfile`
- `ListAllocationOperatingProfileRecords`
- `GetAllocationOperatingProfileTimeline`
- `GetAllocationOperatingProfileEvidence`
- `GetAllocationOperatingProfileDecisionPackage`
- `GetAllocationOperatingProfileExceptions`
- `GetAllocationOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/allocationoperatingprofile/commands
GET  /api/v1/allocationoperatingprofile/{id}
GET  /api/v1/allocationoperatingprofile/{id}/timeline
GET  /api/v1/allocationoperatingprofile/{id}/evidence
GET  /api/v1/allocationoperatingprofile/{id}/decisions
POST /api/v1/allocationoperatingprofile/{id}/exceptions
POST /api/v1/allocationoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`allocationoperatingprofile_aggregate`	authoritative subject
`allocationoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`allocationoperatingprofile_relationship`	governed relationships
`allocationoperatingprofile_decision`	binding Decisions and rationale
`allocationoperatingprofile_evidence_link`	provenance and evidence relationships
`allocationoperatingprofile_event_outbox`	transactional Event publication
`allocationoperatingprofile_idempotency`	duplicate-write protection
`allocationoperatingprofile_exception`	exception, claim, dispute, or hold
`allocationoperatingprofile_correction`	correction and supersession lineage
`allocationoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Allocation` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Pick Pack Ship — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-11-00`
Subject	`PickPackShipDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Pick Pack Ship
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Pick Pack Ship**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Pick Pack Ship**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PickPackShipDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PickPackShipDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePickPackShipDomain`
- `ValidatePickPackShipDomain`
- `ApprovePickPackShipDomain`
- `ActivatePickPackShipDomain`
- `SuspendPickPackShipDomain`
- `CorrectPickPackShipDomain`
- `RetirePickPackShipDomain`

Reference Events:

- `PickPackShipDomainCreated`
- `PickPackShipDomainValidated`
- `PickPackShipDomainApproved`
- `PickPackShipDomainActivated`
- `PickPackShipDomainSuspended`
- `PickPackShipDomainCorrected`
- `PickPackShipDomainRetired`

Reference queries:

- `GetPickPackShipDomain`
- `ListPickPackShipDomainRecords`
- `GetPickPackShipDomainTimeline`
- `GetPickPackShipDomainEvidence`
- `GetPickPackShipDomainDecisionPackage`
- `GetPickPackShipDomainExceptions`
- `GetPickPackShipDomainAuditTrail`

Reference API surface:

```
POST /api/v1/pickpackshipdomain/commands
GET  /api/v1/pickpackshipdomain/{id}
GET  /api/v1/pickpackshipdomain/{id}/timeline
GET  /api/v1/pickpackshipdomain/{id}/evidence
GET  /api/v1/pickpackshipdomain/{id}/decisions
POST /api/v1/pickpackshipdomain/{id}/exceptions
POST /api/v1/pickpackshipdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`pickpackshipdomain_aggregate`	authoritative subject
`pickpackshipdomain_version`	immutable submitted, approved, issued, or signed versions
`pickpackshipdomain_relationship`	governed relationships
`pickpackshipdomain_decision`	binding Decisions and rationale
`pickpackshipdomain_evidence_link`	provenance and evidence relationships
`pickpackshipdomain_event_outbox`	transactional Event publication
`pickpackshipdomain_idempotency`	duplicate-write protection
`pickpackshipdomain_exception`	exception, claim, dispute, or hold
`pickpackshipdomain_correction`	correction and supersession lineage
`pickpackshipdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Pick Pack Ship` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Pick Pack Ship — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-11-01`
Subject	`PickPackShipAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Pick Pack Ship
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Pick Pack Ship**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Pick Pack Ship**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PickPackShipAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PickPackShipAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePickPackShipAggregateModel`
- `ValidatePickPackShipAggregateModel`
- `ApprovePickPackShipAggregateModel`
- `ActivatePickPackShipAggregateModel`
- `SuspendPickPackShipAggregateModel`
- `CorrectPickPackShipAggregateModel`
- `RetirePickPackShipAggregateModel`

Reference Events:

- `PickPackShipAggregateModelCreated`
- `PickPackShipAggregateModelValidated`
- `PickPackShipAggregateModelApproved`
- `PickPackShipAggregateModelActivated`
- `PickPackShipAggregateModelSuspended`
- `PickPackShipAggregateModelCorrected`
- `PickPackShipAggregateModelRetired`

Reference queries:

- `GetPickPackShipAggregateModel`
- `ListPickPackShipAggregateModelRecords`
- `GetPickPackShipAggregateModelTimeline`
- `GetPickPackShipAggregateModelEvidence`
- `GetPickPackShipAggregateModelDecisionPackage`
- `GetPickPackShipAggregateModelExceptions`
- `GetPickPackShipAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/pickpackshipaggregatemodel/commands
GET  /api/v1/pickpackshipaggregatemodel/{id}
GET  /api/v1/pickpackshipaggregatemodel/{id}/timeline
GET  /api/v1/pickpackshipaggregatemodel/{id}/evidence
GET  /api/v1/pickpackshipaggregatemodel/{id}/decisions
POST /api/v1/pickpackshipaggregatemodel/{id}/exceptions
POST /api/v1/pickpackshipaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
pickpackshipaggregatemodel_aggregate	authoritative subject
pickpackshipaggregatemodel_version	immutable submitted, approved, issued, or signed versions
pickpackshipaggregatemodel_relationship	governed relationships
pickpackshipaggregatemodel_decision	binding Decisions and rationale
pickpackshipaggregatemodel_evidence_link	provenance and evidence relationships
pickpackshipaggregatemodel_event_outbox	transactional Event publication
pickpackshipaggregatemodel_idempotency	duplicate-write protection
pickpackshipaggregatemodel_exception	exception, claim, dispute, or hold
pickpackshipaggregatemodel_correction	correction and supersession lineage
pickpackshipaggregatemodel_audit	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Pick Pack Ship` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Pick Pack Ship — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-11-02`
Subject	`PickPackShipContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Pick Pack Ship
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Pick Pack Ship**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Pick Pack Ship**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PickPackShipContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PickPackShipContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePickPackShipContractModel`
- `ValidatePickPackShipContractModel`
- `ApprovePickPackShipContractModel`
- `ActivatePickPackShipContractModel`
- `SuspendPickPackShipContractModel`
- `CorrectPickPackShipContractModel`
- `RetirePickPackShipContractModel`

Reference Events:

- `PickPackShipContractModelCreated`
- `PickPackShipContractModelValidated`
- `PickPackShipContractModelApproved`
- `PickPackShipContractModelActivated`
- `PickPackShipContractModelSuspended`
- `PickPackShipContractModelCorrected`
- `PickPackShipContractModelRetired`

Reference queries:

- `GetPickPackShipContractModel`
- `ListPickPackShipContractModelRecords`
- `GetPickPackShipContractModelTimeline`
- `GetPickPackShipContractModelEvidence`
- `GetPickPackShipContractModelDecisionPackage`
- `GetPickPackShipContractModelExceptions`
- `GetPickPackShipContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/pickpackshipcontractmodel/commands
GET  /api/v1/pickpackshipcontractmodel/{id}
GET  /api/v1/pickpackshipcontractmodel/{id}/timeline
GET  /api/v1/pickpackshipcontractmodel/{id}/evidence
GET  /api/v1/pickpackshipcontractmodel/{id}/decisions
POST /api/v1/pickpackshipcontractmodel/{id}/exceptions
POST /api/v1/pickpackshipcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`pickpackshipcontractmodel_aggregate`	authoritative subject
`pickpackshipcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`pickpackshipcontractmodel_relationship`	governed relationships
`pickpackshipcontractmodel_decision`	binding Decisions and rationale
`pickpackshipcontractmodel_evidence_link`	provenance and evidence relationships
`pickpackshipcontractmodel_event_outbox`	transactional Event publication
`pickpackshipcontractmodel_idempotency`	duplicate-write protection
`pickpackshipcontractmodel_exception`	exception, claim, dispute, or hold
`pickpackshipcontractmodel_correction`	correction and supersession lineage
`pickpackshipcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Pick Pack Ship` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Pick Pack Ship — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	V07-11-03
Subject	PickPackShipDataModel
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	TEMPLATE_DERIVED_REFERENCE_SKELETON unless repository citations prove otherwise
Default implementation classification	REFERENCE_ARCHITECTURE / REQUIRES_REPOSITORY_REVIEW
Accountable owner	Supply Chain Architecture / Pick Pack Ship
Evidence rule	No implementation claim without inspected executable evidence
Repository	chinair88-prog/allinb2c

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Pick Pack Ship**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Pick Pack Ship**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PickPackShipDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PickPackShipDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePickPackShipDataModel`
- `ValidatePickPackShipDataModel`
- `ApprovePickPackShipDataModel`
- `ActivatePickPackShipDataModel`
- `SuspendPickPackShipDataModel`
- `CorrectPickPackShipDataModel`
- `RetirePickPackShipDataModel`

Reference Events:

- `PickPackShipDataModelCreated`
- `PickPackShipDataModelValidated`
- `PickPackShipDataModelApproved`
- `PickPackShipDataModelActivated`
- `PickPackShipDataModelSuspended`
- `PickPackShipDataModelCorrected`
- `PickPackShipDataModelRetired`

Reference queries:

- `GetPickPackShipDataModel`
- `ListPickPackShipDataModelRecords`
- `GetPickPackShipDataModelTimeline`
- `GetPickPackShipDataModelEvidence`
- `GetPickPackShipDataModelDecisionPackage`
- `GetPickPackShipDataModelExceptions`
- `GetPickPackShipDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/pickpackshipdatamodel/commands
GET  /api/v1/pickpackshipdatamodel/{id}
GET  /api/v1/pickpackshipdatamodel/{id}/timeline
GET  /api/v1/pickpackshipdatamodel/{id}/evidence
GET  /api/v1/pickpackshipdatamodel/{id}/decisions
POST /api/v1/pickpackshipdatamodel/{id}/exceptions
POST /api/v1/pickpackshipdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`pickpackshipdatamodel_aggregate`	authoritative subject
`pickpackshipdatamodel_version`	immutable submitted, approved, issued, or signed versions
`pickpackshipdatamodel_relationship`	governed relationships
`pickpackshipdatamodel_decision`	binding Decisions and rationale
`pickpackshipdatamodel_evidence_link`	provenance and evidence relationships
`pickpackshipdatamodel_event_outbox`	transactional Event publication
`pickpackshipdatamodel_idempotency`	duplicate-write protection
`pickpackshipdatamodel_exception`	exception, claim, dispute, or hold
`pickpackshipdatamodel_correction`	correction and supersession lineage
`pickpackshipdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Pick Pack Ship` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Pick Pack Ship — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-11-04`
Subject	`PickPackShipOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Pick Pack Ship
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Pick Pack Ship**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Pick Pack Ship**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`PickPackShipOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`PickPackShipOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
PickPackShipOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreatePickPackShipOperatingProfile`
- `ValidatePickPackShipOperatingProfile`
- `ApprovePickPackShipOperatingProfile`
- `ActivatePickPackShipOperatingProfile`
- `SuspendPickPackShipOperatingProfile`
- `CorrectPickPackShipOperatingProfile`
- `RetirePickPackShipOperatingProfile`

Reference Events:

- `PickPackShipOperatingProfileCreated`
- `PickPackShipOperatingProfileValidated`
- `PickPackShipOperatingProfileApproved`
- `PickPackShipOperatingProfileActivated`
- `PickPackShipOperatingProfileSuspended`
- `PickPackShipOperatingProfileCorrected`
- `PickPackShipOperatingProfileRetired`

Reference queries:

- `GetPickPackShipOperatingProfile`
- `ListPickPackShipOperatingProfileRecords`
- `GetPickPackShipOperatingProfileTimeline`
- `GetPickPackShipOperatingProfileEvidence`
- `GetPickPackShipOperatingProfileDecisionPackage`
- `GetPickPackShipOperatingProfileExceptions`
- `GetPickPackShipOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/pickpackshipoperatingprofile/commands
GET  /api/v1/pickpackshipoperatingprofile/{id}
GET  /api/v1/pickpackshipoperatingprofile/{id}/timeline
GET  /api/v1/pickpackshipoperatingprofile/{id}/evidence
GET  /api/v1/pickpackshipoperatingprofile/{id}/decisions
POST /api/v1/pickpackshipoperatingprofile/{id}/exceptions
POST /api/v1/pickpackshipoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`pickpackshipoperatingprofile_aggregate`	authoritative subject
`pickpackshipoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`pickpackshipoperatingprofile_relationship`	governed relationships
`pickpackshipoperatingprofile_decision`	binding Decisions and rationale
`pickpackshipoperatingprofile_evidence_link`	provenance and evidence relationships
`pickpackshipoperatingprofile_event_outbox`	transactional Event publication
`pickpackshipoperatingprofile_idempotency`	duplicate-write protection
`pickpackshipoperatingprofile_exception`	exception, claim, dispute, or hold
`pickpackshipoperatingprofile_correction`	correction and supersession lineage
`pickpackshipoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Pick Pack Ship` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customs and Border Handling — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-12-00`
Subject	`CustomsandBorderHandlingDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customs and Border Handling
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Customs and Border Handling**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customs and Border Handling**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomsandBorderHandlingDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomsandBorderHandlingDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomsandBorderHandlingDomain`
- `ValidateCustomsandBorderHandlingDomain`
- `ApproveCustomsandBorderHandlingDomain`
- `ActivateCustomsandBorderHandlingDomain`
- `SuspendCustomsandBorderHandlingDomain`
- `CorrectCustomsandBorderHandlingDomain`
- `RetireCustomsandBorderHandlingDomain`

Reference Events:

- `CustomsandBorderHandlingDomainCreated`
- `CustomsandBorderHandlingDomainValidated`
- `CustomsandBorderHandlingDomainApproved`
- `CustomsandBorderHandlingDomainActivated`
- `CustomsandBorderHandlingDomainSuspended`
- `CustomsandBorderHandlingDomainCorrected`
- `CustomsandBorderHandlingDomainRetired`

Reference queries:

- `GetCustomsandBorderHandlingDomain`
- `ListCustomsandBorderHandlingDomainRecords`
- `GetCustomsandBorderHandlingDomainTimeline`
- `GetCustomsandBorderHandlingDomainEvidence`
- `GetCustomsandBorderHandlingDomainDecisionPackage`
- `GetCustomsandBorderHandlingDomainExceptions`
- `GetCustomsandBorderHandlingDomainAuditTrail`

Reference API surface:

```
POST /api/v1/customsandborderhandlingdomain/commands
GET  /api/v1/customsandborderhandlingdomain/{id}
GET  /api/v1/customsandborderhandlingdomain/{id}/timeline
GET  /api/v1/customsandborderhandlingdomain/{id}/evidence
GET  /api/v1/customsandborderhandlingdomain/{id}/decisions
POST /api/v1/customsandborderhandlingdomain/{id}/exceptions
POST /api/v1/customsandborderhandlingdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customsandborderhandlingdomain_aggregate`	authoritative subject
`customsandborderhandlingdomain_version`	immutable submitted, approved, issued, or signed versions
`customsandborderhandlingdomain_relationship`	governed relationships
`customsandborderhandlingdomain_decision`	binding Decisions and rationale
`customsandborderhandlingdomain_evidence_link`	provenance and evidence relationships
`customsandborderhandlingdomain_event_outbox`	transactional Event publication
`customsandborderhandlingdomain_idempotency`	duplicate-write protection
`customsandborderhandlingdomain_exception`	exception, claim, dispute, or hold
`customsandborderhandlingdomain_correction`	correction and supersession lineage
`customsandborderhandlingdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customs and Border Handling` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Customs and Border Handling — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-12-01`
Subject	`CustomsandBorderHandlingAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customs and Border Handling
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Customs and Border Handling**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customs and Border Handling**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomsandBorderHandlingAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomsandBorderHandlingAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomsandBorderHandlingAggregateModel`
- `ValidateCustomsandBorderHandlingAggregateModel`
- `ApproveCustomsandBorderHandlingAggregateModel`
- `ActivateCustomsandBorderHandlingAggregateModel`
- `SuspendCustomsandBorderHandlingAggregateModel`
- `CorrectCustomsandBorderHandlingAggregateModel`
- `RetireCustomsandBorderHandlingAggregateModel`

Reference Events:

- `CustomsandBorderHandlingAggregateModelCreated`
- `CustomsandBorderHandlingAggregateModelValidated`
- `CustomsandBorderHandlingAggregateModelApproved`
- `CustomsandBorderHandlingAggregateModelActivated`
- `CustomsandBorderHandlingAggregateModelSuspended`
- `CustomsandBorderHandlingAggregateModelCorrected`
- `CustomsandBorderHandlingAggregateModelRetired`

Reference queries:

- `GetCustomsandBorderHandlingAggregateModel`
- `ListCustomsandBorderHandlingAggregateModelRecords`
- `GetCustomsandBorderHandlingAggregateModelTimeline`
- `GetCustomsandBorderHandlingAggregateModelEvidence`
- `GetCustomsandBorderHandlingAggregateModelDecisionPackage`
- `GetCustomsandBorderHandlingAggregateModelExceptions`
- `GetCustomsandBorderHandlingAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/customsandborderhandlingaggregatemodel/commands
GET  /api/v1/customsandborderhandlingaggregatemodel/{id}
GET  /api/v1/customsandborderhandlingaggregatemodel/{id}/timeline
GET  /api/v1/customsandborderhandlingaggregatemodel/{id}/evidence
GET  /api/v1/customsandborderhandlingaggregatemodel/{id}/decisions
POST /api/v1/customsandborderhandlingaggregatemodel/{id}/exceptions
POST /api/v1/customsandborderhandlingaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customsandborderhandlingaggregatemodel_aggregate`	authoritative subject
`customsandborderhandlingaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`customsandborderhandlingaggregatemodel_relationship`	governed relationships
`customsandborderhandlingaggregatemodel_decision`	binding Decisions and rationale
`customsandborderhandlingaggregatemodel_evidence_link`	provenance and evidence relationships
`customsandborderhandlingaggregatemodel_event_outbox`	transactional Event publication
`customsandborderhandlingaggregatemodel_idempotency`	duplicate-write protection
`customsandborderhandlingaggregatemodel_exception`	exception, claim, dispute, or hold
`customsandborderhandlingaggregatemodel_correction`	correction and supersession lineage
`customsandborderhandlingaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customs and Border Handling` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customs and Border Handling — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-12-02`
Subject	`CustomsandBorderHandlingContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customs and Border Handling
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Customs and Border Handling**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customs and Border Handling**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomsandBorderHandlingContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomsandBorderHandlingContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomsandBorderHandlingContractModel`
- `ValidateCustomsandBorderHandlingContractModel`
- `ApproveCustomsandBorderHandlingContractModel`
- `ActivateCustomsandBorderHandlingContractModel`
- `SuspendCustomsandBorderHandlingContractModel`
- `CorrectCustomsandBorderHandlingContractModel`
- `RetireCustomsandBorderHandlingContractModel`

Reference Events:

- `CustomsandBorderHandlingContractModelCreated`
- `CustomsandBorderHandlingContractModelValidated`
- `CustomsandBorderHandlingContractModelApproved`
- `CustomsandBorderHandlingContractModelActivated`
- `CustomsandBorderHandlingContractModelSuspended`
- `CustomsandBorderHandlingContractModelCorrected`
- `CustomsandBorderHandlingContractModelRetired`

Reference queries:

- `GetCustomsandBorderHandlingContractModel`
- `ListCustomsandBorderHandlingContractModelRecords`
- `GetCustomsandBorderHandlingContractModelTimeline`
- `GetCustomsandBorderHandlingContractModelEvidence`
- `GetCustomsandBorderHandlingContractModelDecisionPackage`
- `GetCustomsandBorderHandlingContractModelExceptions`
- `GetCustomsandBorderHandlingContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/customsandborderhandlingcontractmodel/commands
GET  /api/v1/customsandborderhandlingcontractmodel/{id}
GET  /api/v1/customsandborderhandlingcontractmodel/{id}/timeline
GET  /api/v1/customsandborderhandlingcontractmodel/{id}/evidence
GET  /api/v1/customsandborderhandlingcontractmodel/{id}/decisions
POST /api/v1/customsandborderhandlingcontractmodel/{id}/exceptions
POST /api/v1/customsandborderhandlingcontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customsandborderhandlingcontractmodel_aggregate`	authoritative subject
`customsandborderhandlingcontractmodel_version`	immutable submitted, approved, issued, or signed versions
`customsandborderhandlingcontractmodel_relationship`	governed relationships
`customsandborderhandlingcontractmodel_decision`	binding Decisions and rationale
`customsandborderhandlingcontractmodel_evidence_link`	provenance and evidence relationships
`customsandborderhandlingcontractmodel_event_outbox`	transactional Event publication
`customsandborderhandlingcontractmodel_idempotency`	duplicate-write protection
`customsandborderhandlingcontractmodel_exception`	exception, claim, dispute, or hold
`customsandborderhandlingcontractmodel_correction`	correction and supersession lineage
`customsandborderhandlingcontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customs and Border Handling` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customs and Border Handling — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-12-03`
Subject	`CustomsandBorderHandlingDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customs and Border Handling
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Customs and Border Handling**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customs and Border Handling**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomsandBorderHandlingDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomsandBorderHandlingDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomsandBorderHandlingDataModel`
- `ValidateCustomsandBorderHandlingDataModel`
- `ApproveCustomsandBorderHandlingDataModel`
- `ActivateCustomsandBorderHandlingDataModel`
- `SuspendCustomsandBorderHandlingDataModel`
- `CorrectCustomsandBorderHandlingDataModel`
- `RetireCustomsandBorderHandlingDataModel`

Reference Events:

- `CustomsandBorderHandlingDataModelCreated`
- `CustomsandBorderHandlingDataModelValidated`
- `CustomsandBorderHandlingDataModelApproved`
- `CustomsandBorderHandlingDataModelActivated`
- `CustomsandBorderHandlingDataModelSuspended`
- `CustomsandBorderHandlingDataModelCorrected`
- `CustomsandBorderHandlingDataModelRetired`

Reference queries:

- `GetCustomsandBorderHandlingDataModel`
- `ListCustomsandBorderHandlingDataModelRecords`
- `GetCustomsandBorderHandlingDataModelTimeline`
- `GetCustomsandBorderHandlingDataModelEvidence`
- `GetCustomsandBorderHandlingDataModelDecisionPackage`
- `GetCustomsandBorderHandlingDataModelExceptions`
- `GetCustomsandBorderHandlingDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/customsandborderhandlingdatamodel/commands
GET  /api/v1/customsandborderhandlingdatamodel/{id}
GET  /api/v1/customsandborderhandlingdatamodel/{id}/timeline
GET  /api/v1/customsandborderhandlingdatamodel/{id}/evidence
GET  /api/v1/customsandborderhandlingdatamodel/{id}/decisions
POST /api/v1/customsandborderhandlingdatamodel/{id}/exceptions
POST /api/v1/customsandborderhandlingdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customsandborderhandlingdatamodel_aggregate`	authoritative subject
`customsandborderhandlingdatamodel_version`	immutable submitted, approved, issued, or signed versions
`customsandborderhandlingdatamodel_relationship`	governed relationships
`customsandborderhandlingdatamodel_decision`	binding Decisions and rationale
`customsandborderhandlingdatamodel_evidence_link`	provenance and evidence relationships
`customsandborderhandlingdatamodel_event_outbox`	transactional Event publication
`customsandborderhandlingdatamodel_idempotency`	duplicate-write protection
`customsandborderhandlingdatamodel_exception`	exception, claim, dispute, or hold
`customsandborderhandlingdatamodel_correction`	correction and supersession lineage
`customsandborderhandlingdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customs and Border Handling` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Customs and Border Handling — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-12-04`
Subject	`CustomsandBorderHandlingOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Customs and Border Handling
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Customs and Border Handling**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Customs and Border Handling**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`CustomsandBorderHandlingOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`CustomsandBorderHandlingOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
CustomsandBorderHandlingOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateCustomsandBorderHandlingOperatingProfile`
- `ValidateCustomsandBorderHandlingOperatingProfile`
- `ApproveCustomsandBorderHandlingOperatingProfile`
- `ActivateCustomsandBorderHandlingOperatingProfile`
- `SuspendCustomsandBorderHandlingOperatingProfile`
- `CorrectCustomsandBorderHandlingOperatingProfile`
- `RetireCustomsandBorderHandlingOperatingProfile`

Reference Events:

- `CustomsandBorderHandlingOperatingProfileCreated`
- `CustomsandBorderHandlingOperatingProfileValidated`
- `CustomsandBorderHandlingOperatingProfileApproved`
- `CustomsandBorderHandlingOperatingProfileActivated`
- `CustomsandBorderHandlingOperatingProfileSuspended`
- `CustomsandBorderHandlingOperatingProfileCorrected`
- `CustomsandBorderHandlingOperatingProfileRetired`

Reference queries:

- `GetCustomsandBorderHandlingOperatingProfile`
- `ListCustomsandBorderHandlingOperatingProfileRecords`
- `GetCustomsandBorderHandlingOperatingProfileTimeline`
- `GetCustomsandBorderHandlingOperatingProfileEvidence`
- `GetCustomsandBorderHandlingOperatingProfileDecisionPackage`
- `GetCustomsandBorderHandlingOperatingProfileExceptions`
- `GetCustomsandBorderHandlingOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/customsandborderhandlingoperatingprofile/commands
GET  /api/v1/customsandborderhandlingoperatingprofile/{id}
GET  /api/v1/customsandborderhandlingoperatingprofile/{id}/timeline
GET  /api/v1/customsandborderhandlingoperatingprofile/{id}/evidence
GET  /api/v1/customsandborderhandlingoperatingprofile/{id}/decisions
POST /api/v1/customsandborderhandlingoperatingprofile/{id}/exceptions
POST /api/v1/customsandborderhandlingoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`customsandborderhandlingoperatingprofile_aggregate`	authoritative subject
`customsandborderhandlingoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`customsandborderhandlingoperatingprofile_relationship`	governed relationships
`customsandborderhandlingoperatingprofile_decision`	binding Decisions and rationale
`customsandborderhandlingoperatingprofile_evidence_link`	provenance and evidence relationships
`customsandborderhandlingoperatingprofile_event_outbox`	transactional Event publication
`customsandborderhandlingoperatingprofile_idempotency`	duplicate-write protection
`customsandborderhandlingoperatingprofile_exception`	exception, claim, dispute, or hold
`customsandborderhandlingoperatingprofile_correction`	correction and supersession lineage
`customsandborderhandlingoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Customs and Border Handling` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.

Status	Meaning
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Exceptions and Claims — Orientation and Evidence Baseline

Metadata	Value
Volume	7
Chapter ID	`V07-13-00`
Subject	`ExceptionsandClaimsDomain`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Exceptions and Claims
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define the scope, non-goals, module and database evidence requirements, ownership boundaries, current status, and target responsibilities of Exceptions and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Exceptions and Claims**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ExceptionsandClaimsDomain`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDomainVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDomainDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDomainEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDomainException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ExceptionsandClaimsDomain {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateExceptionsandClaimsDomain`
- `ValidateExceptionsandClaimsDomain`
- `ApproveExceptionsandClaimsDomain`
- `ActivateExceptionsandClaimsDomain`
- `SuspendExceptionsandClaimsDomain`
- `CorrectExceptionsandClaimsDomain`
- `RetireExceptionsandClaimsDomain`

Reference Events:

- `ExceptionsandClaimsDomainCreated`
- `ExceptionsandClaimsDomainValidated`
- `ExceptionsandClaimsDomainApproved`
- `ExceptionsandClaimsDomainActivated`
- `ExceptionsandClaimsDomainSuspended`
- `ExceptionsandClaimsDomainCorrected`
- `ExceptionsandClaimsDomainRetired`

Reference queries:

- `GetExceptionsandClaimsDomain`
- `ListExceptionsandClaimsDomainRecords`
- `GetExceptionsandClaimsDomainTimeline`
- `GetExceptionsandClaimsDomainEvidence`
- `GetExceptionsandClaimsDomainDecisionPackage`
- `GetExceptionsandClaimsDomainExceptions`
- `GetExceptionsandClaimsDomainAuditTrail`

Reference API surface:

```
POST /api/v1/exceptionsandclaimsdomain/commands
GET  /api/v1/exceptionsandclaimsdomain/{id}
GET  /api/v1/exceptionsandclaimsdomain/{id}/timeline
GET  /api/v1/exceptionsandclaimsdomain/{id}/evidence
GET  /api/v1/exceptionsandclaimsdomain/{id}/decisions
POST /api/v1/exceptionsandclaimsdomain/{id}/exceptions
POST /api/v1/exceptionsandclaimsdomain/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`exceptionsandclaimsdomain_aggregate`	authoritative subject
`exceptionsandclaimsdomain_version`	immutable submitted, approved, issued, or signed versions
`exceptionsandclaimsdomain_relationship`	governed relationships
`exceptionsandclaimsdomain_decision`	binding Decisions and rationale
`exceptionsandclaimsdomain_evidence_link`	provenance and evidence relationships
`exceptionsandclaimsdomain_event_outbox`	transactional Event publication
`exceptionsandclaimsdomain_idempotency`	duplicate-write protection
`exceptionsandclaimsdomain_exception`	exception, claim, dispute, or hold
`exceptionsandclaimsdomain_correction`	correction and supersession lineage
`exceptionsandclaimsdomain_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Exceptions and Claims` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.

Exceptions and Claims — Ubiquitous Language, Aggregates, and State

Metadata	Value
Volume	7
Chapter ID	`V07-13-01`
Subject	`ExceptionsandClaimsAggregateModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Exceptions and Claims
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define entities, value objects, relationships, aggregates, invariants, lifecycle state, effective time, correction, and concurrency for Exceptions and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Exceptions and Claims**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ExceptionsandClaimsAggregateModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsAggregateModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsAggregateModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsAggregateModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsAggregateModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ExceptionsandClaimsAggregateModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateExceptionsandClaimsAggregateModel`
- `ValidateExceptionsandClaimsAggregateModel`
- `ApproveExceptionsandClaimsAggregateModel`
- `ActivateExceptionsandClaimsAggregateModel`
- `SuspendExceptionsandClaimsAggregateModel`
- `CorrectExceptionsandClaimsAggregateModel`
- `RetireExceptionsandClaimsAggregateModel`

Reference Events:

- `ExceptionsandClaimsAggregateModelCreated`
- `ExceptionsandClaimsAggregateModelValidated`
- `ExceptionsandClaimsAggregateModelApproved`
- `ExceptionsandClaimsAggregateModelActivated`
- `ExceptionsandClaimsAggregateModelSuspended`
- `ExceptionsandClaimsAggregateModelCorrected`
- `ExceptionsandClaimsAggregateModelRetired`

Reference queries:

- `GetExceptionsandClaimsAggregateModel`
- `ListExceptionsandClaimsAggregateModelRecords`
- `GetExceptionsandClaimsAggregateModelTimeline`
- `GetExceptionsandClaimsAggregateModelEvidence`
- `GetExceptionsandClaimsAggregateModelDecisionPackage`
- `GetExceptionsandClaimsAggregateModelExceptions`
- `GetExceptionsandClaimsAggregateModelAuditTrail`

Reference API surface:

```
POST /api/v1/exceptionsandclaimsaggregatemodel/commands
GET  /api/v1/exceptionsandclaimsaggregatemodel/{id}
GET  /api/v1/exceptionsandclaimsaggregatemodel/{id}/timeline
GET  /api/v1/exceptionsandclaimsaggregatemodel/{id}/evidence
GET  /api/v1/exceptionsandclaimsaggregatemodel/{id}/decisions
POST /api/v1/exceptionsandclaimsaggregatemodel/{id}/exceptions
POST /api/v1/exceptionsandclaimsaggregatemodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`exceptionsandclaimsaggregatemodel_aggregate`	authoritative subject
`exceptionsandclaimsaggregatemodel_version`	immutable submitted, approved, issued, or signed versions
`exceptionsandclaimsaggregatemodel_relationship`	governed relationships
`exceptionsandclaimsaggregatemodel_decision`	binding Decisions and rationale
`exceptionsandclaimsaggregatemodel_evidence_link`	provenance and evidence relationships
`exceptionsandclaimsaggregatemodel_event_outbox`	transactional Event publication
`exceptionsandclaimsaggregatemodel_idempotency`	duplicate-write protection
`exceptionsandclaimsaggregatemodel_exception`	exception, claim, dispute, or hold
`exceptionsandclaimsaggregatemodel_correction`	correction and supersession lineage
`exceptionsandclaimsaggregatemodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Exceptions and Claims` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Exceptions and Claims — Commands, Events, APIs, Workflows, and Tools

Metadata	Value
Volume	7
Chapter ID	`V07-13-02`
Subject	`ExceptionsandClaimsContractModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Exceptions and Claims
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define typed Commands, completed-fact Events, REST/gRPC/BFF contracts, long-running workflows, compensations, provider adapters, and AI Tools for Exceptions and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Exceptions and Claims**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ExceptionsandClaimsContractModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsContractModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsContractModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsContractModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsContractModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ExceptionsandClaimsContractModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateExceptionsandClaimsContractModel`
- `ValidateExceptionsandClaimsContractModel`
- `ApproveExceptionsandClaimsContractModel`
- `ActivateExceptionsandClaimsContractModel`
- `SuspendExceptionsandClaimsContractModel`
- `CorrectExceptionsandClaimsContractModel`
- `RetireExceptionsandClaimsContractModel`

Reference Events:

- `ExceptionsandClaimsContractModelCreated`
- `ExceptionsandClaimsContractModelValidated`
- `ExceptionsandClaimsContractModelApproved`
- `ExceptionsandClaimsContractModelActivated`
- `ExceptionsandClaimsContractModelSuspended`
- `ExceptionsandClaimsContractModelCorrected`
- `ExceptionsandClaimsContractModelRetired`

Reference queries:

- `GetExceptionsandClaimsContractModel`
- `ListExceptionsandClaimsContractModelRecords`
- `GetExceptionsandClaimsContractModelTimeline`
- `GetExceptionsandClaimsContractModelEvidence`
- `GetExceptionsandClaimsContractModelDecisionPackage`
- `GetExceptionsandClaimsContractModelExceptions`
- `GetExceptionsandClaimsContractModelAuditTrail`

Reference API surface:

```
POST /api/v1/exceptionsandclaimscontractmodel/commands
GET  /api/v1/exceptionsandclaimscontractmodel/{id}
GET  /api/v1/exceptionsandclaimscontractmodel/{id}/timeline
GET  /api/v1/exceptionsandclaimscontractmodel/{id}/evidence
GET  /api/v1/exceptionsandclaimscontractmodel/{id}/decisions
POST /api/v1/exceptionsandclaimscontractmodel/{id}/exceptions
POST /api/v1/exceptionsandclaimscontractmodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`exceptionsandclaimscontractmodel_aggregate`	authoritative subject
`exceptionsandclaimscontractmodel_version`	immutable submitted, approved, issued, or signed versions
`exceptionsandclaimscontractmodel_relationship`	governed relationships
`exceptionsandclaimscontractmodel_decision`	binding Decisions and rationale
`exceptionsandclaimscontractmodel_evidence_link`	provenance and evidence relationships
`exceptionsandclaimscontractmodel_event_outbox`	transactional Event publication
`exceptionsandclaimscontractmodel_idempotency`	duplicate-write protection
`exceptionsandclaimscontractmodel_exception`	exception, claim, dispute, or hold
`exceptionsandclaimscontractmodel_correction`	correction and supersession lineage
`exceptionsandclaimscontractmodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Exceptions and Claims` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Exceptions and Claims — Data, Tables, Migrations, Lineage, and Reconciliation

Metadata	Value
Volume	7
Chapter ID	`V07-13-03`
Subject	`ExceptionsandClaimsDataModel`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Exceptions and Claims
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define database ownership, schemas, table families, keys, constraints, indexes, migrations, lineage, projections, reconciliation, retention, backup, and correction for Exceptions and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Exceptions and Claims**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ExceptionsandClaimsDataModel`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDataModelVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDataModelDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDataModelEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsDataModelException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ExceptionsandClaimsDataModel {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateExceptionsandClaimsDataModel`
- `ValidateExceptionsandClaimsDataModel`
- `ApproveExceptionsandClaimsDataModel`
- `ActivateExceptionsandClaimsDataModel`
- `SuspendExceptionsandClaimsDataModel`
- `CorrectExceptionsandClaimsDataModel`
- `RetireExceptionsandClaimsDataModel`

Reference Events:

- `ExceptionsandClaimsDataModelCreated`
- `ExceptionsandClaimsDataModelValidated`
- `ExceptionsandClaimsDataModelApproved`
- `ExceptionsandClaimsDataModelActivated`
- `ExceptionsandClaimsDataModelSuspended`
- `ExceptionsandClaimsDataModelCorrected`
- `ExceptionsandClaimsDataModelRetired`

Reference queries:

- `GetExceptionsandClaimsDataModel`
- `ListExceptionsandClaimsDataModelRecords`
- `GetExceptionsandClaimsDataModelTimeline`
- `GetExceptionsandClaimsDataModelEvidence`
- `GetExceptionsandClaimsDataModelDecisionPackage`
- `GetExceptionsandClaimsDataModelExceptions`
- `GetExceptionsandClaimsDataModelAuditTrail`

Reference API surface:

```
POST /api/v1/exceptionsandclaimsdatamodel/commands
GET  /api/v1/exceptionsandclaimsdatamodel/{id}
GET  /api/v1/exceptionsandclaimsdatamodel/{id}/timeline
GET  /api/v1/exceptionsandclaimsdatamodel/{id}/evidence
GET  /api/v1/exceptionsandclaimsdatamodel/{id}/decisions
POST /api/v1/exceptionsandclaimsdatamodel/{id}/exceptions
POST /api/v1/exceptionsandclaimsdatamodel/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`exceptionsandclaimsdatamodel_aggregate`	authoritative subject
`exceptionsandclaimsdatamodel_version`	immutable submitted, approved, issued, or signed versions
`exceptionsandclaimsdatamodel_relationship`	governed relationships
`exceptionsandclaimsdatamodel_decision`	binding Decisions and rationale
`exceptionsandclaimsdatamodel_evidence_link`	provenance and evidence relationships
`exceptionsandclaimsdatamodel_event_outbox`	transactional Event publication
`exceptionsandclaimsdatamodel_idempotency`	duplicate-write protection
`exceptionsandclaimsdatamodel_exception`	exception, claim, dispute, or hold
`exceptionsandclaimsdatamodel_correction`	correction and supersession lineage
`exceptionsandclaimsdatamodel_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Exceptions and Claims` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Exceptions and Claims — Portals, AI, Security, SLOs, Tests, and Operations

Metadata	Value
Volume	7
Chapter ID	`V07-13-04`
Subject	`ExceptionsandClaimsOperatingProfile`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Architecture / Exceptions and Claims
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **define Portal journeys, AI participation, trust boundaries, SLOs, runbooks, tests, migration, examples, and conformance for Exceptions and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Architecture / Exceptions and Claims**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;
- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`ExceptionsandClaimsOperatingProfile`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsOperatingProfileVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsOperatingProfileDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsOperatingProfileEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`ExceptionsandClaimsOperatingProfileException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
ExceptionsandClaimsOperatingProfile {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions

- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage
- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateExceptionsandClaimsOperatingProfile`
- `ValidateExceptionsandClaimsOperatingProfile`
- `ApproveExceptionsandClaimsOperatingProfile`
- `ActivateExceptionsandClaimsOperatingProfile`
- `SuspendExceptionsandClaimsOperatingProfile`
- `CorrectExceptionsandClaimsOperatingProfile`
- `RetireExceptionsandClaimsOperatingProfile`

Reference Events:

- `ExceptionsandClaimsOperatingProfileCreated`
- `ExceptionsandClaimsOperatingProfileValidated`
- `ExceptionsandClaimsOperatingProfileApproved`
- `ExceptionsandClaimsOperatingProfileActivated`
- `ExceptionsandClaimsOperatingProfileSuspended`
- `ExceptionsandClaimsOperatingProfileCorrected`
- `ExceptionsandClaimsOperatingProfileRetired`

Reference queries:

- `GetExceptionsandClaimsOperatingProfile`
- `ListExceptionsandClaimsOperatingProfileRecords`
- `GetExceptionsandClaimsOperatingProfileTimeline`
- `GetExceptionsandClaimsOperatingProfileEvidence`
- `GetExceptionsandClaimsOperatingProfileDecisionPackage`
- `GetExceptionsandClaimsOperatingProfileExceptions`
- `GetExceptionsandClaimsOperatingProfileAuditTrail`

Reference API surface:

```
POST /api/v1/exceptionsandclaimsoperatingprofile/commands
GET  /api/v1/exceptionsandclaimsoperatingprofile/{id}
GET  /api/v1/exceptionsandclaimsoperatingprofile/{id}/timeline
GET  /api/v1/exceptionsandclaimsoperatingprofile/{id}/evidence
GET  /api/v1/exceptionsandclaimsoperatingprofile/{id}/decisions
POST /api/v1/exceptionsandclaimsoperatingprofile/{id}/exceptions
POST /api/v1/exceptionsandclaimsoperatingprofile/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`exceptionsandclaimsoperatingprofile_aggregate`	authoritative subject
`exceptionsandclaimsoperatingprofile_version`	immutable submitted, approved, issued, or signed versions
`exceptionsandclaimsoperatingprofile_relationship`	governed relationships
`exceptionsandclaimsoperatingprofile_decision`	binding Decisions and rationale
`exceptionsandclaimsoperatingprofile_evidence_link`	provenance and evidence relationships
`exceptionsandclaimsoperatingprofile_event_outbox`	transactional Event publication
`exceptionsandclaimsoperatingprofile_idempotency`	duplicate-write protection
`exceptionsandclaimsoperatingprofile_exception`	exception, claim, dispute, or hold
`exceptionsandclaimsoperatingprofile_correction`	correction and supersession lineage
`exceptionsandclaimsoperatingprofile_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions
- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- Named capability `Exceptions and Claims` is part of the target handbook scope.
- Exact service paths, APIs, tables, Events and tests must be inspected before VERIFIED_IMPLEMENTED status.
- Known overlaps and duplicate ownership are classified as CONFLICT_REQUIRES_CANONICALIZATION.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	`REFERENCE_ARCHITECTURE`
registered module or catalog claim	`DOCUMENTATION_ONLY_CLAIM` until source inspection
uninspected integration	`REQUIRES_REPOSITORY_REVIEW`
behavior proven in another repository-grounded chapter	retain that chapter's evidence and classification
contradictory ownership or path claims	`CONFLICT_REQUIRES_CANONICALIZATION`

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
`VERIFIED_IMPLEMENTED`	Executable source and relevant behavior were inspected.
`IMPLEMENTED_WITH_GAP`	Executable behavior exists with a material governance or completeness gap.
`PARTIAL_IMPLEMENTATION`	Only part of the target capability is evidenced.
`REFERENCE_ARCHITECTURE`	Normative target behavior; no claim that it exists today.
`PLANNED_NOT_IMPLEMENTED`	Explicitly planned but not evidenced as executable.
`DOCUMENTATION_ONLY_CLAIM`	Documentation or client contract claims behavior without verified backend evidence.
`CONFLICT_REQUIRES_CANONICALIZATION`	Sources disagree and no final owner has been established.
`REQUIRES_REPOSITORY_REVIEW`	Necessary executable evidence has not yet been inspected.

Fulfilment and Supply Chain Domain Map

Metadata	Value
Volume	7
Chapter ID	`V07-SYNTH`
Subject	`SupplyChainDomainMap`
Status	Generated First-Draft Architecture Skeleton - Domain-Specific Expansion Required
Content maturity	`TEMPLATE_DERIVED_REFERENCE_SKELETON` unless repository citations prove otherwise
Default implementation classification	`REFERENCE_ARCHITECTURE` / `REQUIRES_REPOSITORY_REVIEW`
Accountable owner	Supply Chain Council
Evidence rule	No implementation claim without inspected executable evidence
Repository	`chinair88-prog/allinb2c`

---

1. Executive Orientation

This chapter defines how GTOS shall **connect Supplier, Provider, Customer, CRM, Inventory, Warehouse, Shipping, Regulatory, Fulfilment, Allocation, Customs, and Claims**.

The specification separates reality, observations, knowledge, Decisions, Intent, execution, and outcomes. A component name, database row, UI label, AI answer, or workflow state does not acquire authority merely because it is visible or technically convenient.

```
Reality or external outcome
    ≠ Observation
    ≠ Claim
    ≠ derived Perspective
    ≠ Prediction
    ≠ Recommendation
    ≠ authorized Decision
    ≠ execution Intent
```

1.1 Accountable ownership

The accountable owner is **Supply Chain Council**. Supporting applications, shared infrastructure, integration adapters, analytics, and AI coordinate or present the capability but shall not silently own its authoritative propositions.

1.2 Scope

- create and identify the subject
- validate evidence and policy
- make authorized Decisions
- execute typed Commands
- publish completed-fact Events
- query current and historical perspectives
- correct and reconcile outcomes

1.3 Non-goals

- treating a registered Gradle module, catalog record, README, frontend route, migration plan, or generated report as proof of end-to-end implementation;

- direct cross-Domain database writes;
- generic `status` values that combine lifecycle, approval, provider, document, payment, dispute, and correction state;
- silent replacement of history;
- AI-created authority or fabricated external outcomes.

---

2. Ubiquitous Language and Subject Model

Subject	Required interpretation
`SupplyChainDomainMap`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplyChainDomainMapVersion`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplyChainDomainMapDecision`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplyChainDomainMapEvidenceSet`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit
`SupplyChainDomainMapException`	Independent identified subject, governed subordinate entity, or qualified perspective; ownership must be explicit

Reference aggregate:

```
SupplyChainDomainMap {
  subjectId
  tenantId
  organizationContext
  sourceIdentityRefs[]
  sourceVersionRefs[]
  lifecycleState
  relationshipRefs[]
  decisionRefs[]
  intentRefs[]
  evidenceSetRef
  knowledgeCutoff
  policyRefs[]
  authorityRef
  jurisdictionRefs[]
  createdAt
  effectiveAt
  updatedAt
  version
  correctionOf?
  correlationId
}
```

2.1 Core invariants

- Stable identity precedes relationship, state, and cross-system correlation.
- Effective time and recorded time remain distinct where business truth changes over time.
- Recommendation, Decision, Intent, execution, and outcome are separate concepts.
- No Portal, report, search index, cache, workflow, or AI Agent becomes a shadow owner of Domain truth.
- Binding submitted, approved, issued, accepted, or signed versions are immutable.
- Corrections append governed facts and preserve the original record.
- External authority and provider outcomes retain their source and qualification.
- Tenant and represented-principal context come from trusted authentication and mandate evaluation.
- Retried writes are idempotent and concurrency guarded.
- Outcome Unknown is explicit and reconciled before risky duplicate execution.

2.2 Required standards

- stable identifiers
- typed contracts
- immutable binding versions
- transactional outbox
- idempotent Commands
- explicit Outcome Unknown
- correction lineage

- machine-verifiable conformance

---

3. Actors, Roles, and Authority

A conformant design distinguishes:

- the natural person acting;
- the service or AI identity invoking a Tool;
- the organization or principal represented;
- the role or relationship granting context;
- the mandate or delegation granting action authority;
- the Policy and jurisdiction limiting the action;
- the reviewer or approver responsible for a binding Decision.

Authority is evaluated at action time. Authentication alone does not grant business authority. Cached permissions and browser-supplied tenant headers are insufficient for high-impact actions.

---

4. Lifecycle and State Model

State	Semantics
`PROPOSED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`VALIDATION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`APPROVAL_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`ACTIVE`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`SUSPENDED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`CORRECTION_PENDING`	Distinct governed condition with entry, exit, timeout, correction and authority semantics
`RETIRED`	Distinct governed condition with entry, exit, timeout, correction and authority semantics

```
stateDiagram-v2
    [*] --> PROPOSED
    PROPOSED --> VALIDATION_PENDING
    VALIDATION_PENDING --> APPROVAL_PENDING
    APPROVAL_PENDING --> ACTIVE
    ACTIVE --> SUSPENDED
    SUSPENDED --> CORRECTION_PENDING
    CORRECTION_PENDING --> RETIRED
```

The diagram is a primary reading path. Real implementations may add parallel review, amendment, appeal, timeout, suspension, partial completion, reconciliation, correction, and terminal failure without collapsing their meaning.

4.1 Transition rules

- transition through a typed, authorized Command;
- require exact expected version where concurrent change matters;
- record actor, represented principal, mandate, Policy, reason, and Evidence;
- use an idempotency key for retried writes;
- publish a completed-fact Event after durable state change;
- expose Outcome Unknown for unresolved external completion;
- preserve original and corrected states in historical reconstruction.

---

5. Decision, Evidence, and Knowledge Cutoff

Reference Decision:

```
Decision {
  decisionId
```

```
  decisionType
  subjectRef
  sourceVersionRefs[]
  knowledgeCutoff
  evidenceSetRef
  policyVersionRefs[]
  recommendationRefs[]
  principalId
  representedOrganizationId
  mandateRef
  rationale
  outcome
  decidedAt
  effectiveAt
  correctionOf?
}
```

Evidence records identify issuer or observer, subject, version, observed time, received time, effective interval, integrity, confidence, qualification, contradiction, correction, retention, residency, and access class. A recommendation may assist the Decision but cannot promote itself to the Decision.

---

6. Commands, Events, Queries, and Contracts

Reference Commands:

- `CreateSupplyChainDomainMap`
- `ValidateSupplyChainDomainMap`
- `ApproveSupplyChainDomainMap`
- `ActivateSupplyChainDomainMap`
- `SuspendSupplyChainDomainMap`
- `CorrectSupplyChainDomainMap`
- `RetireSupplyChainDomainMap`

Reference Events:

- `SupplyChainDomainMapCreated`
- `SupplyChainDomainMapValidated`
- `SupplyChainDomainMapApproved`
- `SupplyChainDomainMapActivated`
- `SupplyChainDomainMapSuspended`
- `SupplyChainDomainMapCorrected`
- `SupplyChainDomainMapRetired`

Reference queries:

- `GetSupplyChainDomainMap`
- `ListSupplyChainDomainMapRecords`
- `GetSupplyChainDomainMapTimeline`
- `GetSupplyChainDomainMapEvidence`
- `GetSupplyChainDomainMapDecisionPackage`
- `GetSupplyChainDomainMapExceptions`
- `GetSupplyChainDomainMapAuditTrail`

Reference API surface:

```
POST /api/v1/supplychaindomainmap/commands
GET  /api/v1/supplychaindomainmap/{id}
GET  /api/v1/supplychaindomainmap/{id}/timeline
GET  /api/v1/supplychaindomainmap/{id}/evidence
GET  /api/v1/supplychaindomainmap/{id}/decisions
POST /api/v1/supplychaindomainmap/{id}/exceptions
POST /api/v1/supplychaindomainmap/{id}/corrections
```

These endpoints are reference contracts, not claims that exact routes exist.

---

7. Workflow, Failure, and Compensation

```
flowchart TD
    A[Validate identity version and authority] --> B[Collect required evidence]
    B --> C{Policy and evidence gates pass?}
    C -- No --> D[Open exception or request correction]
    D --> B
```

```
    C -- Yes --> E[Record Decision or execution Intent]
    E --> F[Execute Domain command or external request]
    F --> G{Outcome known?}
    G -- Success --> H[Record fact and publish Event]
    G -- Failure --> I[Record failure and compensation options]
    G -- Unknown --> J[Enter Outcome Unknown]
    J --> K[Query source or wait for verified callback]
    K --> G
    H --> L[Update projections and downstream workflow]
```

Representative failure modes:

- stale or wrong source version
- duplicate submission or replay
- external timeout after possible success
- contradictory source observations
- lost Event after committed state
- approval or mandate revoked during execution
- downstream action before a mandatory gate
- partial completion across multiple subjects
- evidence later retracted or corrected
- tenant, facility, channel or jurisdiction mismatch

Recovery shall query the authoritative source before duplicate execution, preserve partial success, use compensating Intent rather than destructive rewrite, publish corrections, quarantine high-impact ambiguity, and record affected downstream subjects.

---

8. Data and Persistence Architecture

Reference table families:

Table family	Purpose
`supplychaindomainmap_aggregate`	authoritative subject
`supplychaindomainmap_version`	immutable submitted, approved, issued, or signed versions
`supplychaindomainmap_relationship`	governed relationships
`supplychaindomainmap_decision`	binding Decisions and rationale
`supplychaindomainmap_evidence_link`	provenance and evidence relationships
`supplychaindomainmap_event_outbox`	transactional Event publication
`supplychaindomainmap_idempotency`	duplicate-write protection
`supplychaindomainmap_exception`	exception, claim, dispute, or hold
`supplychaindomainmap_correction`	correction and supersession lineage
`supplychaindomainmap_audit`	privileged action evidence

The owning service controls schema and migration. Cross-Domain references use stable identifiers or contracts. Analytics, reports, caches, search, and vector indexes remain projections. Backup, restore, retention, legal hold, privacy, residency, and decommissioning requirements are explicit.

---

9. Portal and Experience Requirements

- Primary participant Portal: initiate, review exact versions, supply evidence, and respond to required actions.
- Operator Portal: inspect exceptions, correlation, source responses, retries, compensation, and downstream impact.
- Executive or oversight view: consume governed metrics without becoming a source of Domain truth.

Every Portal displays exact version, owner, state, freshness, required action, and uncertainty. It supports loading, empty, partial, stale, denied, error, timeout, and Outcome Unknown states; accessibility; locale, currency, unit, date, and RTL/LTR correctness; evidence audit; safe retry; and correction history.

---

10. AI Participation and Tool Governance

Permitted AI tasks:

- extract and classify evidence
- detect anomalies and missing fields
- summarize timelines and contradictions

- recommend next actions with sources and uncertainty

AI shall not own Domain records, grant itself authority, suppress contradictions, silently edit approved versions, fabricate external outcomes, or execute high-impact actions outside typed Tools, policy, approval, idempotency, sandbox, and outcome observation.

Every Agent Run records source context, prompt/model/tool versions, structured output, confidence and limitations, Policy result, approval, Tool calls, provider responses, final outcome, and recovery actions.

---

11. Security, Privacy, and Trust Boundaries

- least privilege and separation of duties
- tenant, organization, facility, channel and jurisdiction isolation
- integrity protection for binding evidence
- verified external callbacks and anti-replay controls

Additional controls include secret isolation, callback signature verification, audit of privileged action, emergency-access review, sensitive-data minimization, purpose limitation, and threat models for impersonation, tampering, replay, confused deputy, fraud, prompt injection, and exfiltration.

---

12. Reliability, SLOs, and Operations

SLI	Reference objective
authoritative read availability	99.9% monthly unless a stricter tier applies
acknowledged Command durability	no acknowledged write may be silently lost
Event publication	99.9% within five minutes after committed fact
duplicate prevention	100% for same tenant, Command class, and idempotency key
binding Decision audit completeness	100%
stale-state detection	bounded by business deadline

Dashboards and runbooks cover backlog, error budget, dependency failure, retry budget, DLQ, workflow stall, provider outage, reconciliation, capacity, backup/restore, release/rollback, incident command, and post-incident correction.

---

13. Testing and Continuous Conformance

- aggregate invariant and transition tests
- authorization, delegation, separation-of-duties and tenant-isolation tests
- idempotency and optimistic-concurrency tests
- database migration, key, constraint and rollback tests
- contract, outbox/inbox, duplicate, replay and DLQ tests
- workflow restart, timeout and compensation tests
- Portal E2E tests for success, partial, stale, denied, error and Outcome Unknown states
- accessibility, locale, currency, unit and RTL/LTR tests
- security adversarial, fraud and data-exfiltration tests
- AI evaluation, prompt-injection, policy-bypass and unauthorized-tool tests
- historical reconstruction, correction and audit-completeness tests
- performance, capacity, recovery and operational exercise tests

Release evidence links each invariant and control to the test, environment, input data, result, owner, and artifact proving it. Generated test counts never replace semantic coverage.

---

14. Repository Review Boundary

- This chapter is a repository-review specification. Exact executable implementation has not been inspected for every target behavior.
- Registered modules, catalog entries, README claims, frontend contracts, or plans are not proof of complete implementation.

Area	Classification
target aggregate, API, tables, state and workflow in this chapter	REFERENCE_ARCHITECTURE
registered module or catalog claim	DOCUMENTATION_ONLY_CLAIM until source inspection
uninspected integration	REQUIRES_REPOSITORY_REVIEW
behavior proven in another repository-grounded chapter	retain that chapter’s evidence and classification
contradictory ownership or path claims	CONFLICT_REQUIRES_CANONICALIZATION

No statement upgrades implementation status merely because a file, directory, module name, frontend contract, database entry, plan, or report exists.

---

15. Worked Conformance Scenario

An authorized principal initiates a Command against an exact subject version. The service validates identity, tenant, mandate, Policy, Evidence, and idempotency. It records the state transition and outbox entry atomically. If an external dependency participates, timeout produces Outcome Unknown; reconciliation queries the source or waits for a verified callback instead of repeating the action blindly.

Portals show source, freshness, version, required action, and uncertainty. AI may extract, compare, summarize, detect anomalies, and recommend through typed Tools, but cannot own the subject or fabricate success. Later correction appends new facts, identifies affected projections and workflows, and preserves the historical Decision context.

---

16. Conformance Checklist

- ☐ stable identity and proposition ownership are explicit;
- ☐ authority and mandate are evaluated at action time;
- ☐ state, Recommendation, Decision, Intent, execution, and outcome remain distinct;
- ☐ Commands are typed, authorized, idempotent, and version guarded;
- ☐ Events describe completed facts;
- ☐ Evidence includes provenance, time, integrity, qualification, and correction;
- ☐ external outcomes retain their authority source;
- ☐ Portals show freshness and uncertainty;
- ☐ AI is bounded by typed Tools, Policy, approval, and outcome observation;
- ☐ security, tenant, facility, channel, and jurisdiction boundaries are enforced;
- ☐ Outcome Unknown and reconciliation exist where needed;
- ☐ corrections preserve history;
- ☐ tests prove invariants and recovery;
- ☐ implementation status is not upgraded without executable evidence.

---

17. Status Vocabulary

Status	Meaning
VERIFIED_IMPLEMENTED	Executable source and relevant behavior were inspected.
IMPLEMENTED_WITH_GAP	Executable behavior exists with a material governance or completeness gap.
PARTIAL_IMPLEMENTATION	Only part of the target capability is evidenced.
REFERENCE_ARCHITECTURE	Normative target behavior; no claim that it exists today.
PLANNED_NOT_IMPLEMENTED	Explicitly planned but not evidenced as executable.
DOCUMENTATION_ONLY_CLAIM	Documentation or client contract claims behavior without verified backend evidence.
CONFLICT_REQUIRES_CANONICALIZATION	Sources disagree and no final owner has been established.
REQUIRES_REPOSITORY_REVIEW	Necessary executable evidence has not yet been inspected.